<u>SDITO - Lab</u>

<u>Name:</u> Sanjib Ghara **<u>Roll No:</u>** CSE/23/065 **<u>Exam Roll No</u>**.11600223092

<u>Assignment 11</u>

<u>Problem Statement :</u> Build scaling plans in AWS that balance the load on different EC2 instances.

<u>Aim</u>

To configure a Launch Template, create an Auto Scaling Group (ASG) integrated with an Application Load Balancer (ALB), and test both fault tolerance (instance termination) and scale-out policies (CPU stress).

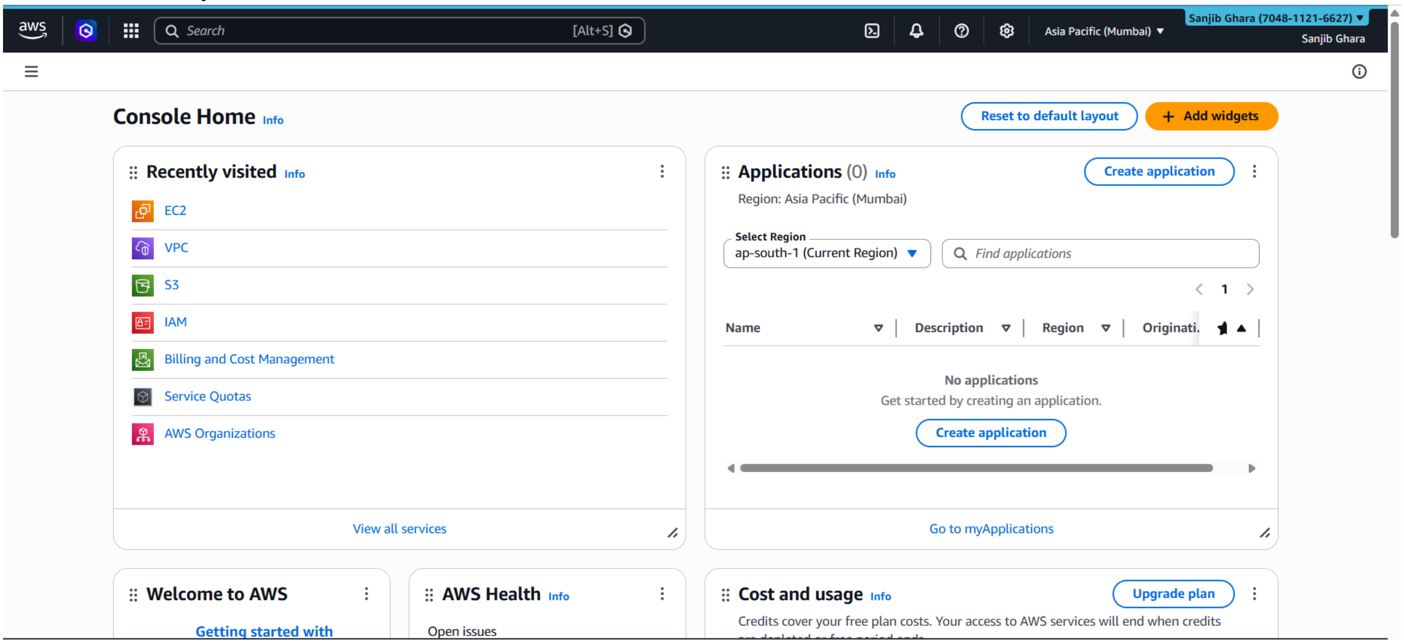
<u>Architecture used</u>

- EC2 Instances
- Target Group
- Application Load Balancer (ALB)
- Auto Scaling Group (ASG)
- Scaling Policy

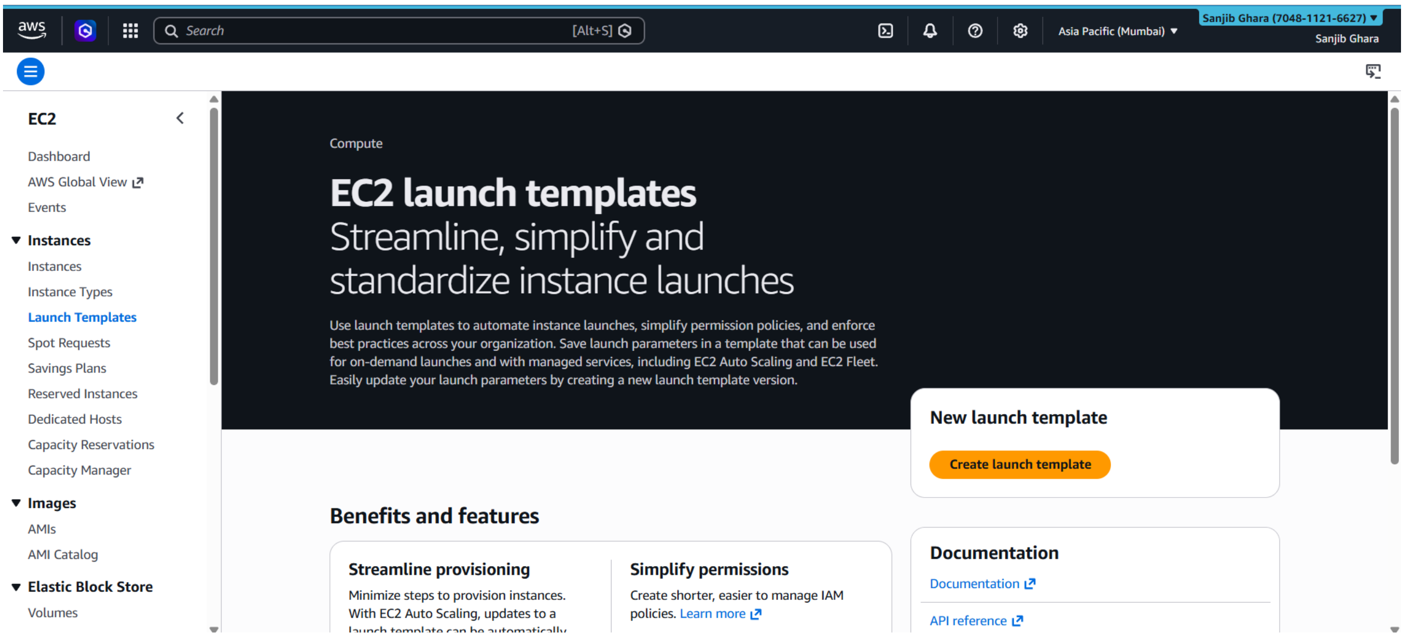
<u>Process:</u>

<u>Phase 1: Create a Launch Template</u>

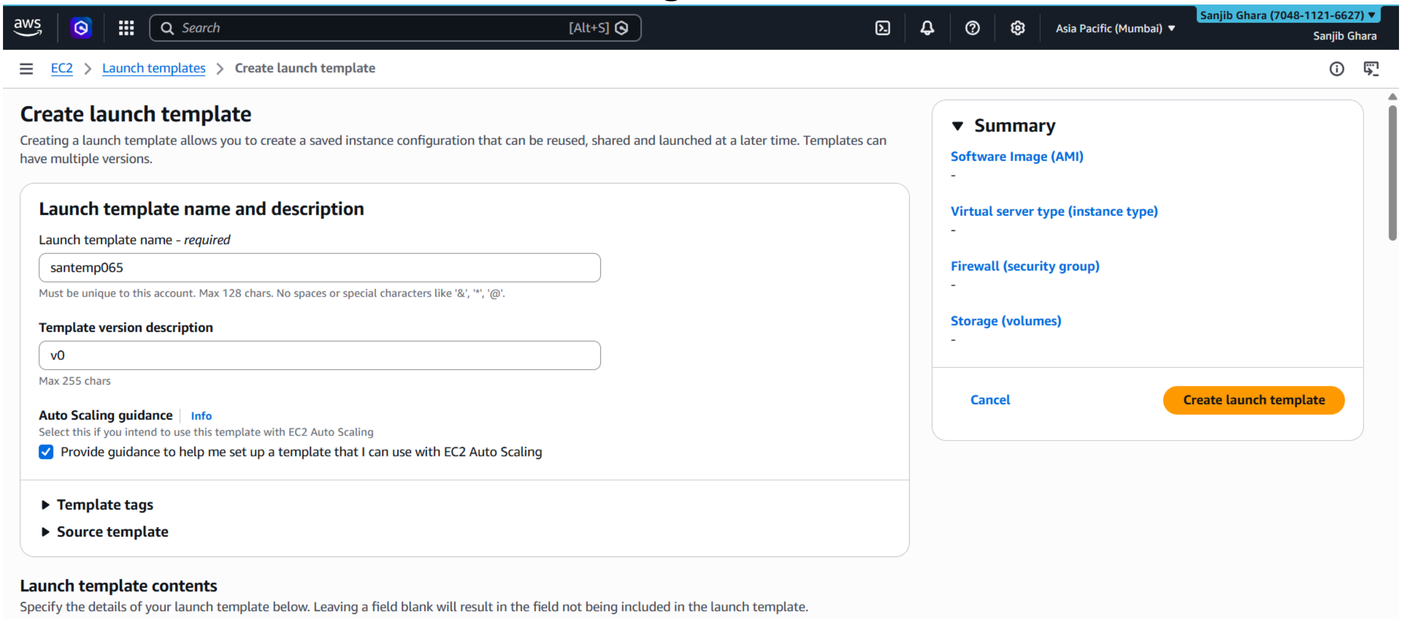
1. **Navigate to EC2 Dashboard:** Log in to the AWS Management Console and open the EC2 console.



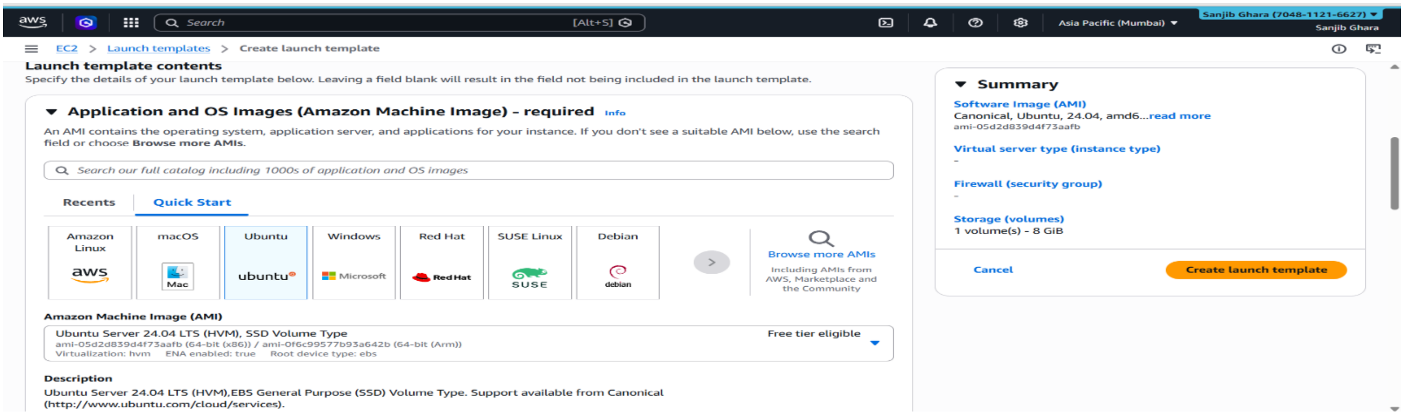
2. **Access Launch Templates:** In the left navigation pane, under **Instances**, select **Launch Templates**, then click **Create launch template**.



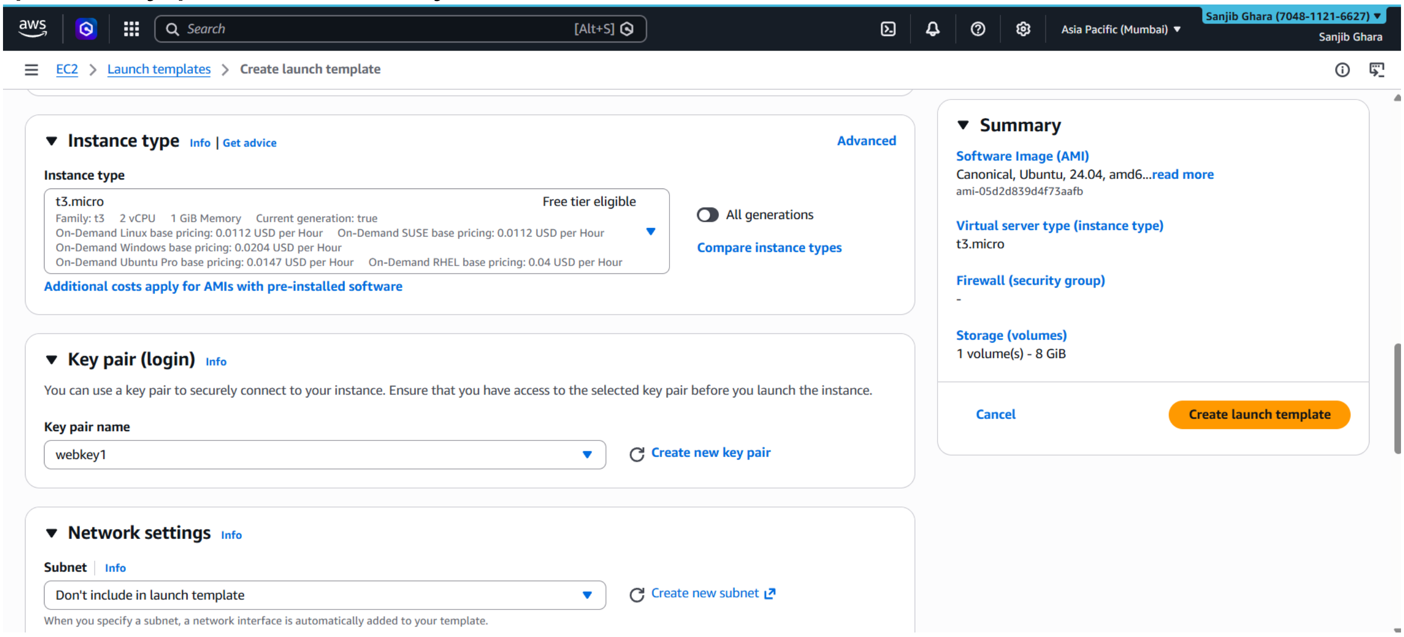
3. **Template Details:**
- **Name:** Enter santemp065.
- **Template version description:** v0
- Check the box for **Provide guidance to help set up a template that I can use with EC2 Auto Scaling**.



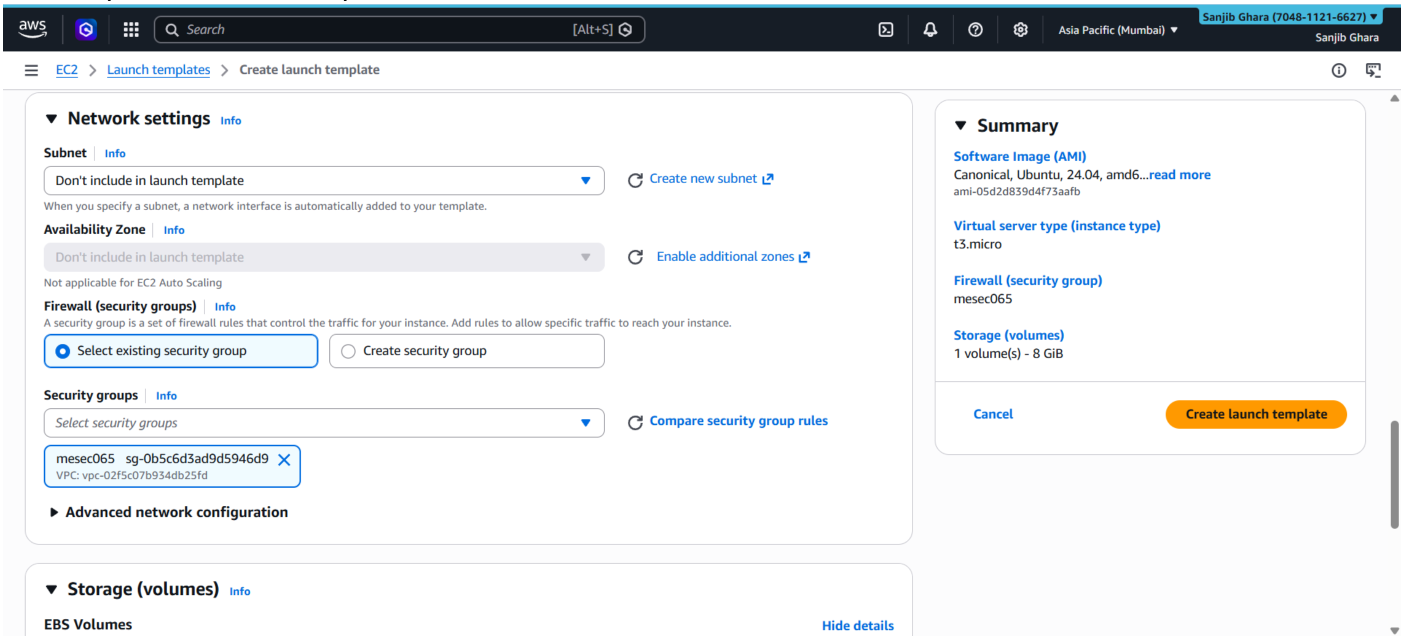
4. **Application and OS Images (AMI):** Search for and select **Ubuntu Server 24.04 LTS (HVM)**.



5. **Instance Type:** Select **t3.micro** (Free tier eligible).
6. **Key Pair:** Select an existing Key Pair (e.g., webkey1) or create a new RSA .pem key pair to ensure you can SSH into the instances later.



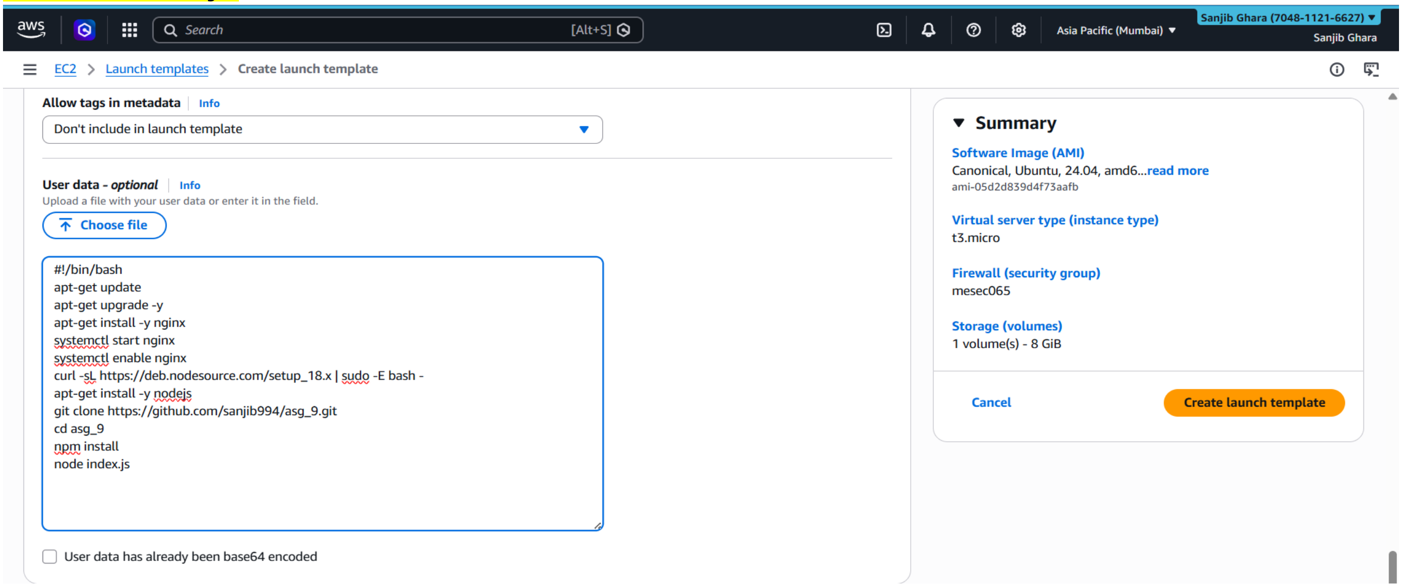
7. **Network Settings:**
   - **Subnet:** Leave this as "Don't include in launch template" (subnets will be defined in the ASG).
   - **Firewall (Security Groups):** Select an existing security group (e.g., mykey1) that allows inbound traffic on port 22 (SSH) and port 4000 (Custom TCP).



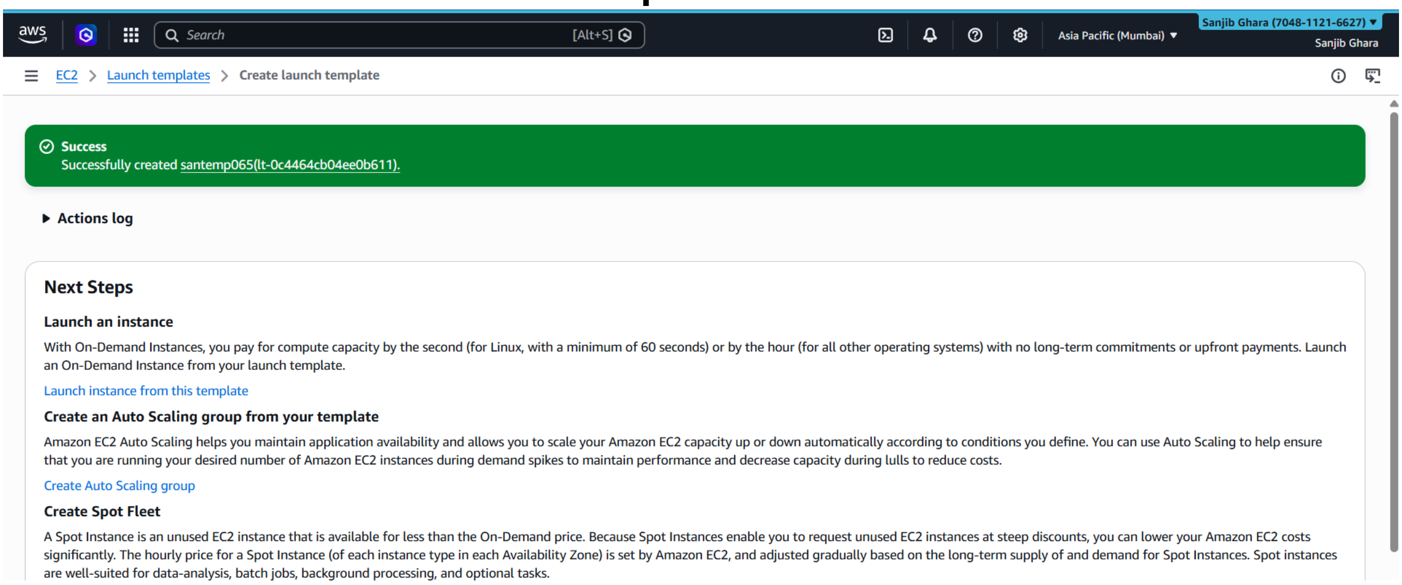
8. **Advanced Details (User Data):** Scroll down to the User data field and paste the following bash script to automatically install necessary packages and start your Node.js application upon boot:

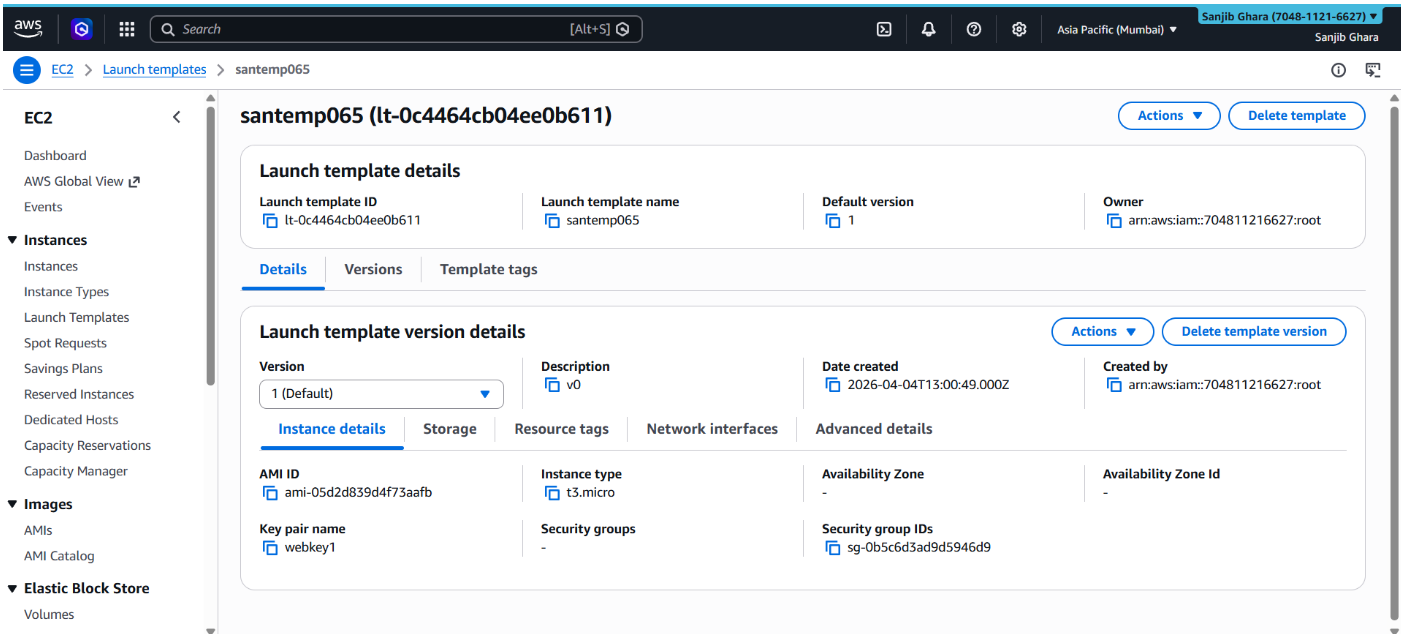
Bash

```
#!/bin/bash
apt-get update
apt-get upgrade -y
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
git clone https://github.com/sanjib994/asg_9.git
cd awslab
npm install
node index.js
```



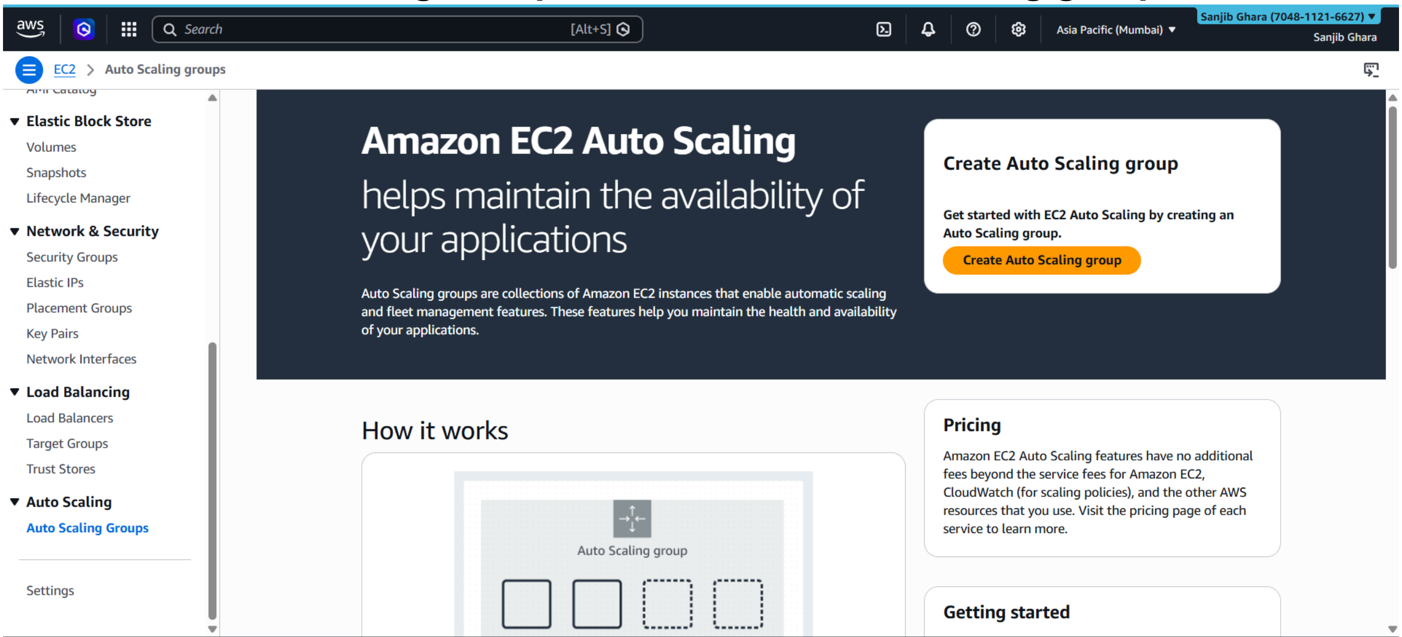
9. **Create:** Click **Create launch template**.



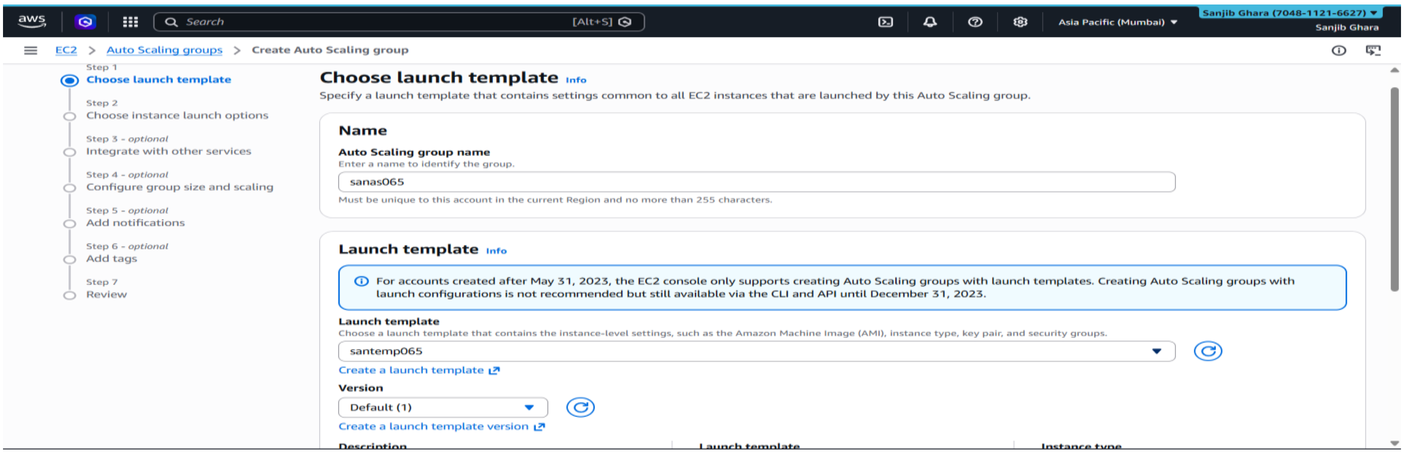


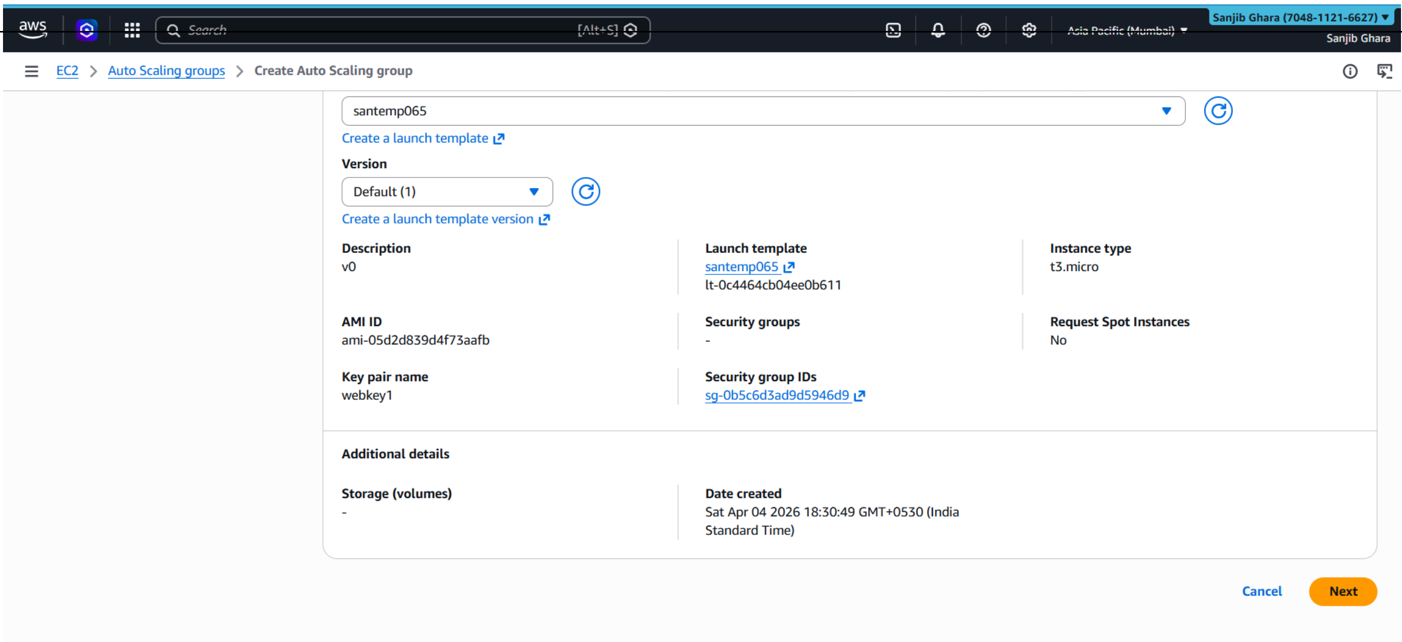
Phase 2: Create the Auto Scaling Group (ASG)

1. **Navigate to Auto Scaling:** In the left panel, scroll down to **Auto Scaling** and select **Auto Scaling Groups**. Click **Create Auto Scaling group**.

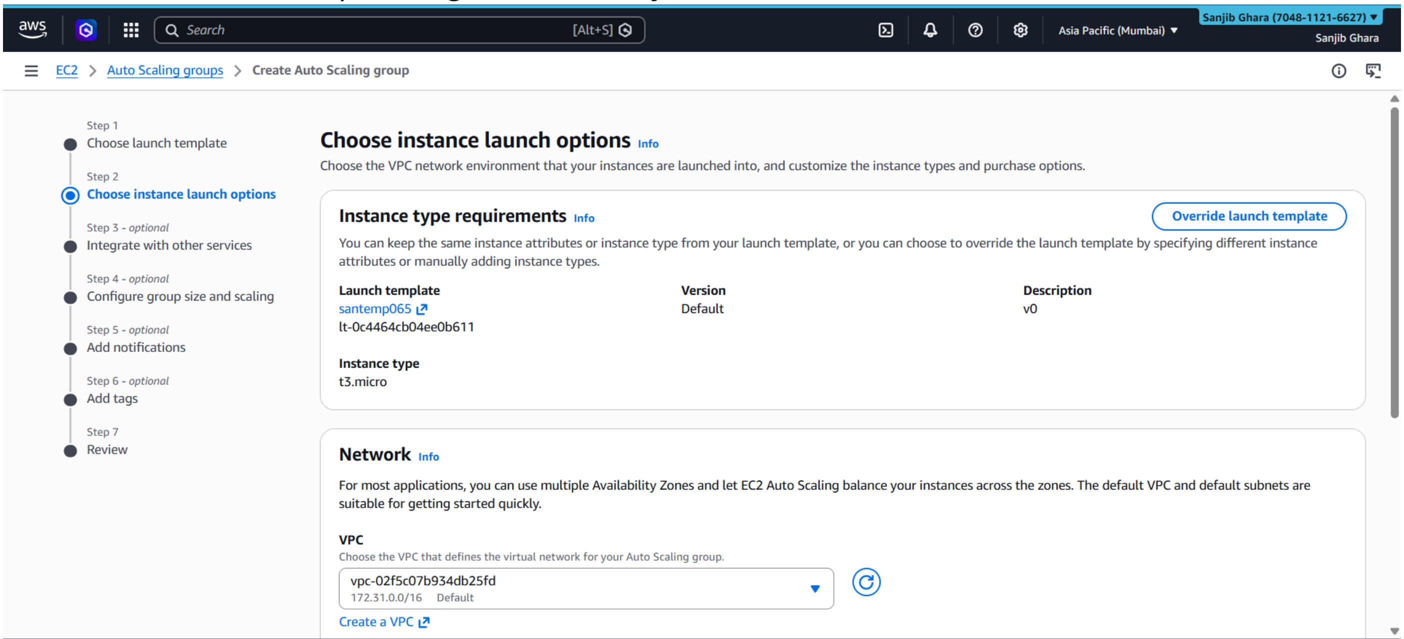


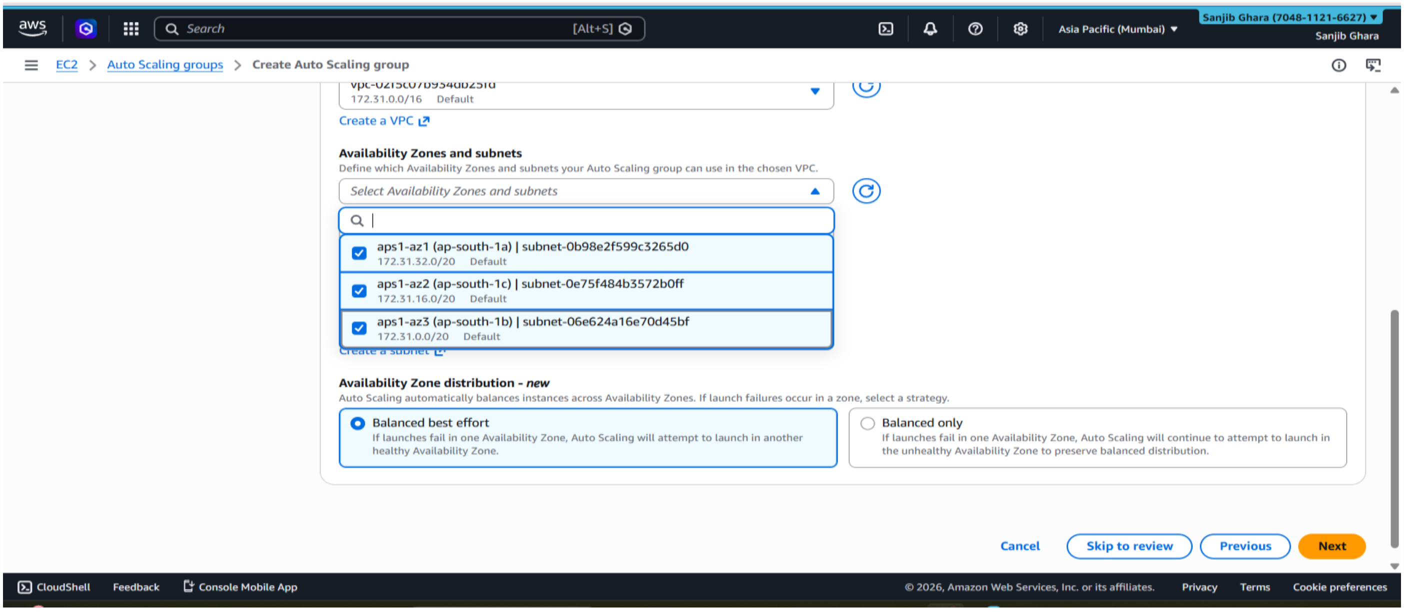
2. **Step 1: Choose Launch Template:** Name the AutoScalingGroup sanas065.
   - Select the santemp065 your just created Launch template. Click **Next**.



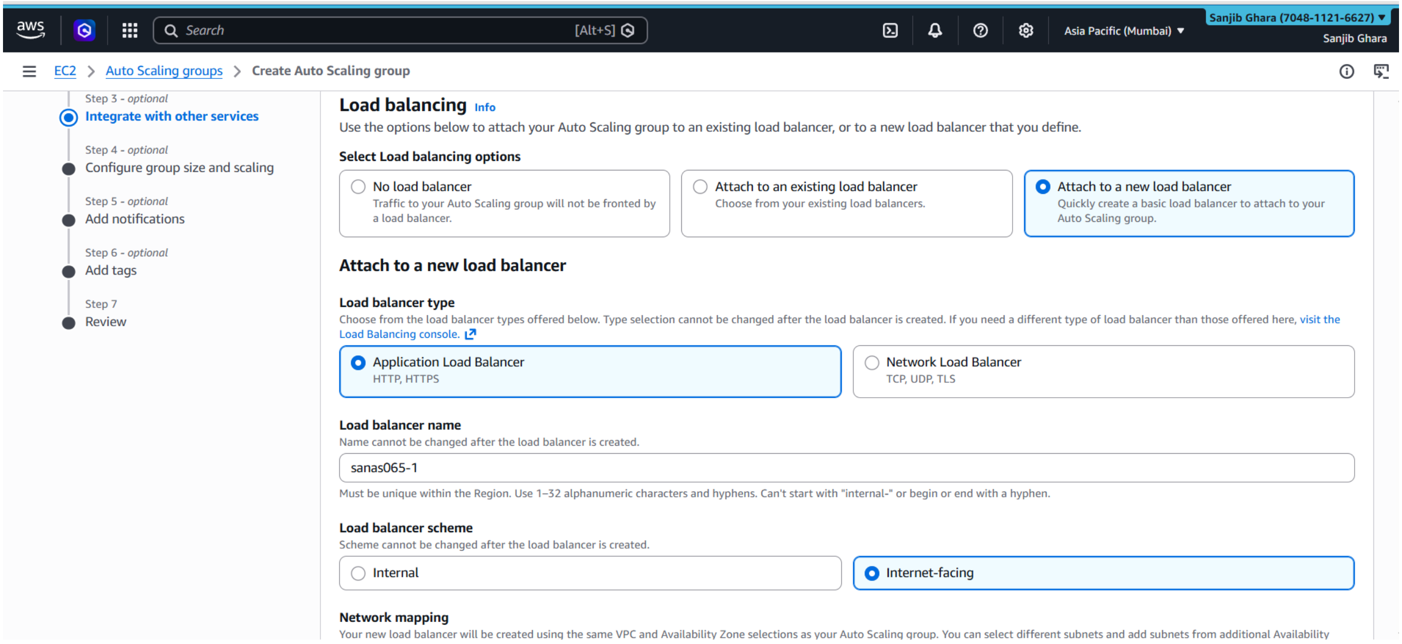


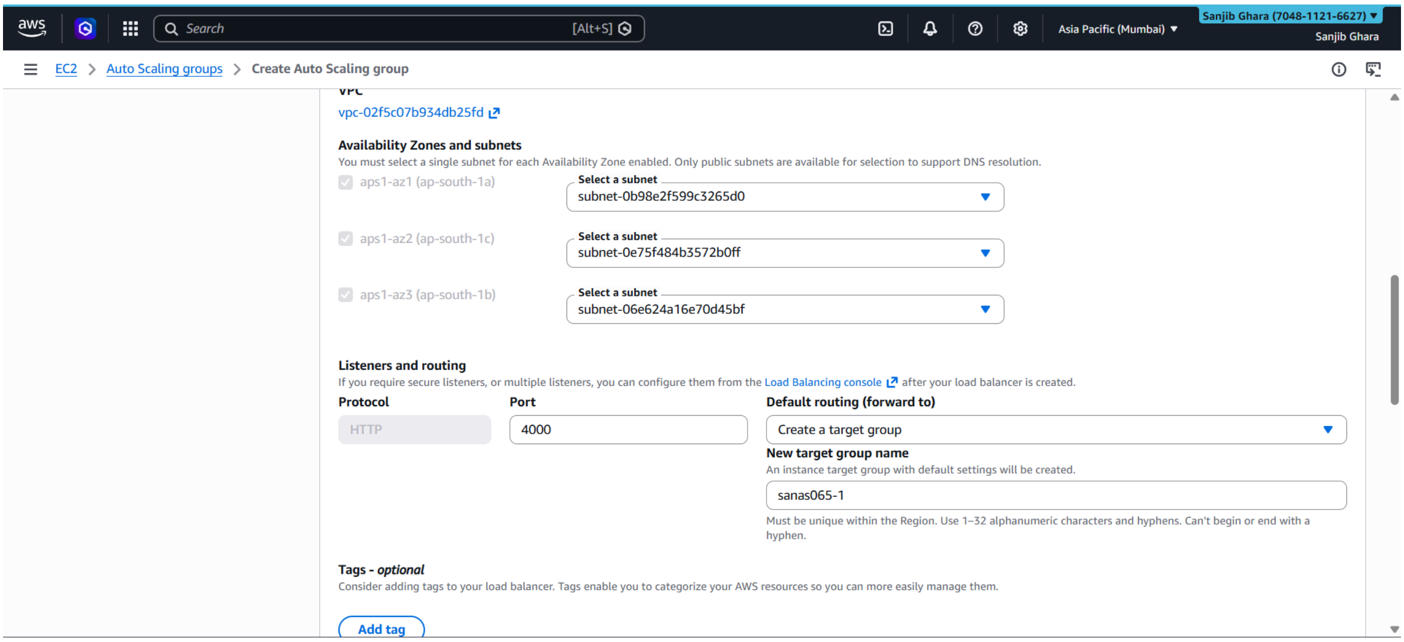
3. **Step 2: Choose Instance Launch Options:**
   - Select your default **VPC**.
   - Select multiple **Availability Zones** (e.g., ap-south-1a, ap-south-1b, ap-south-1c) for high availability. Click **Next**.

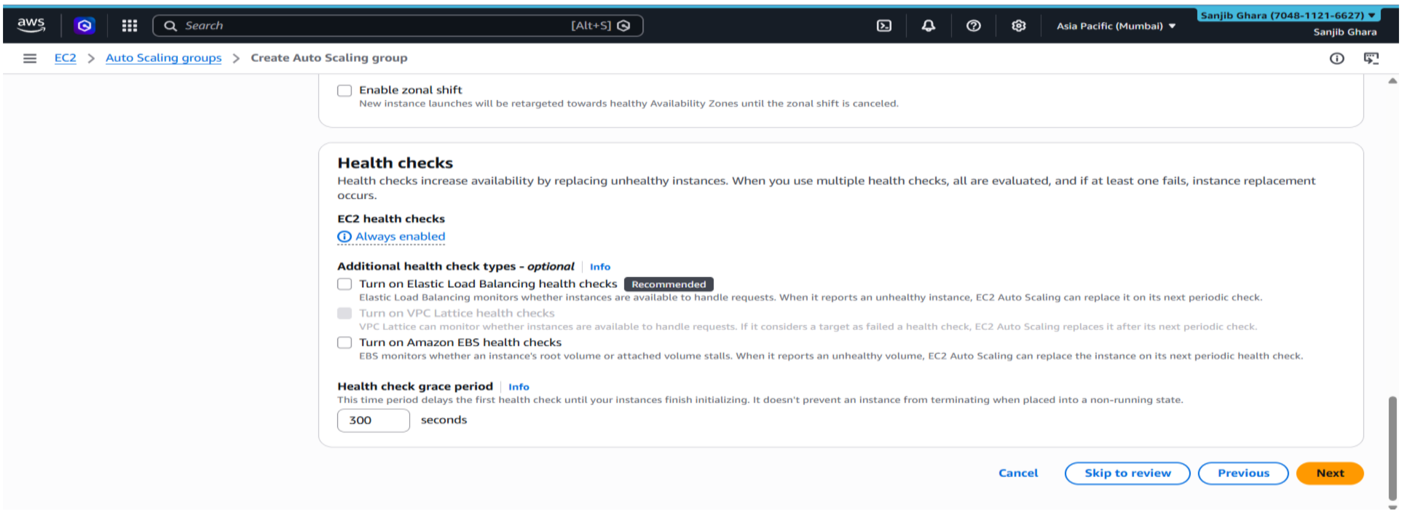




4. **Step 3: Integrate with Other Services (Load Balancing):**
   - Select **Attach to a new load balancer**.
   - Choose **Application Load Balancer** and set the scheme to **Internet-facing**.
   - Under Listeners and routing, ensure the protocol is HTTP and set the Port to **4000**. Select **Create a target group** and name it
     → sanas065-1.
   - Under Health Checks, set Health check grace period to 300 seconds . Click **Next**.

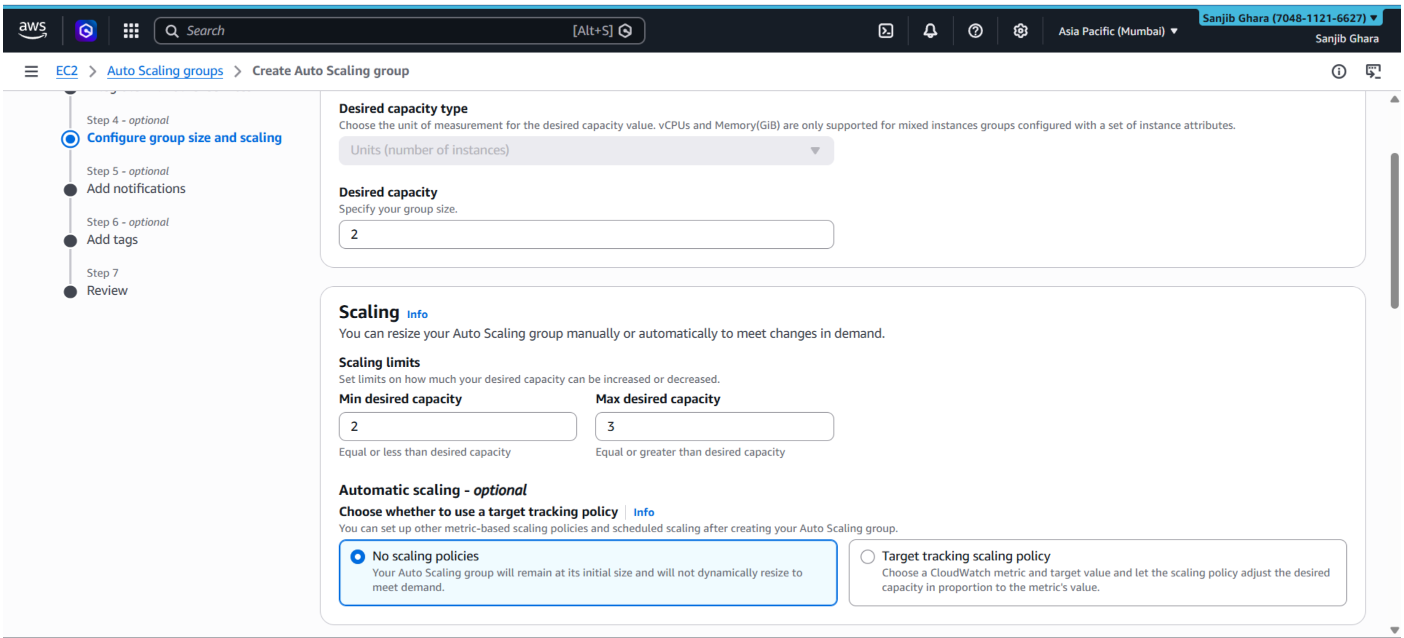


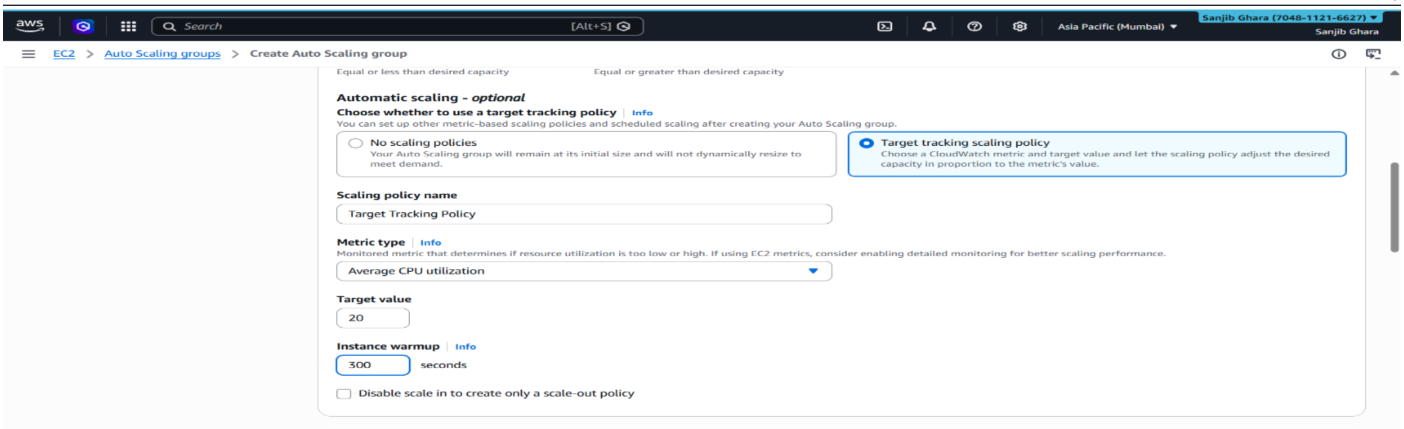


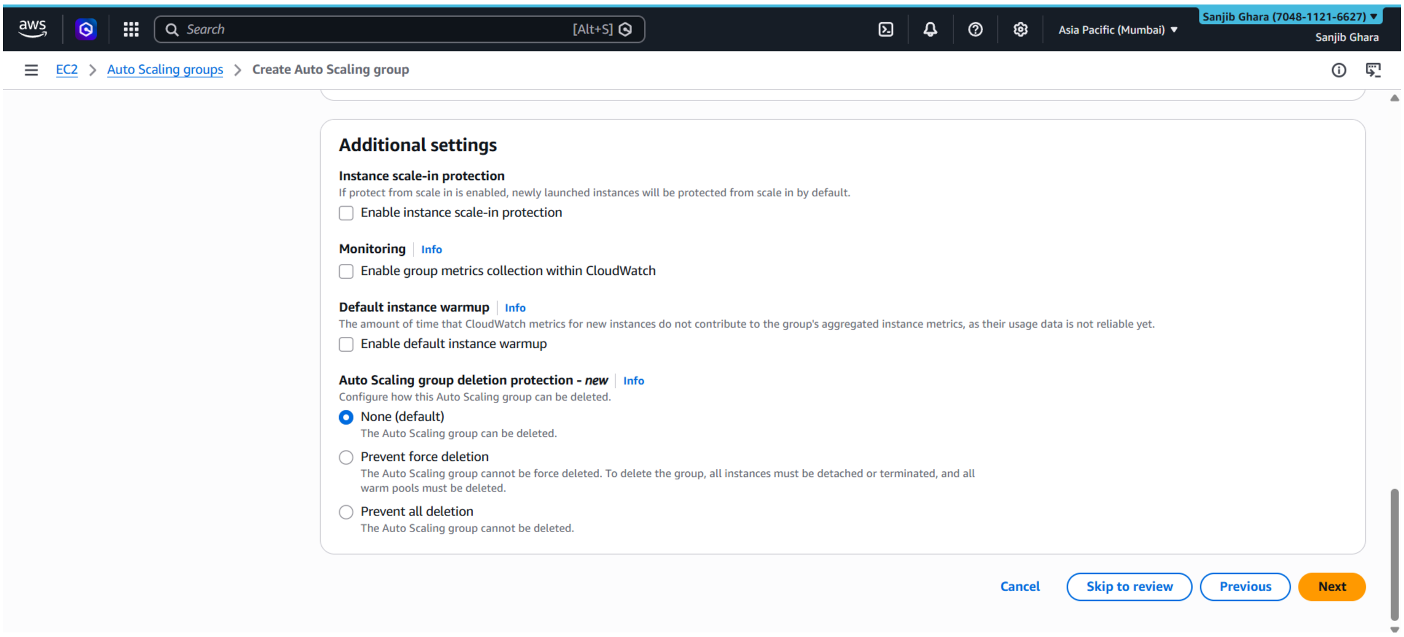


5. **Step 4: Configure Group Size and Scaling:**
   - Set **Desired capacity** to 2, **Minimum capacity** to 2, and **Maximum capacity** to 3.
   - Select **Target tracking scaling policy**.
   - Metric type: **Average CPU utilization**.
   - Target value: **20** (This means if average CPU usage goes above 20%, it will launch a new instance).

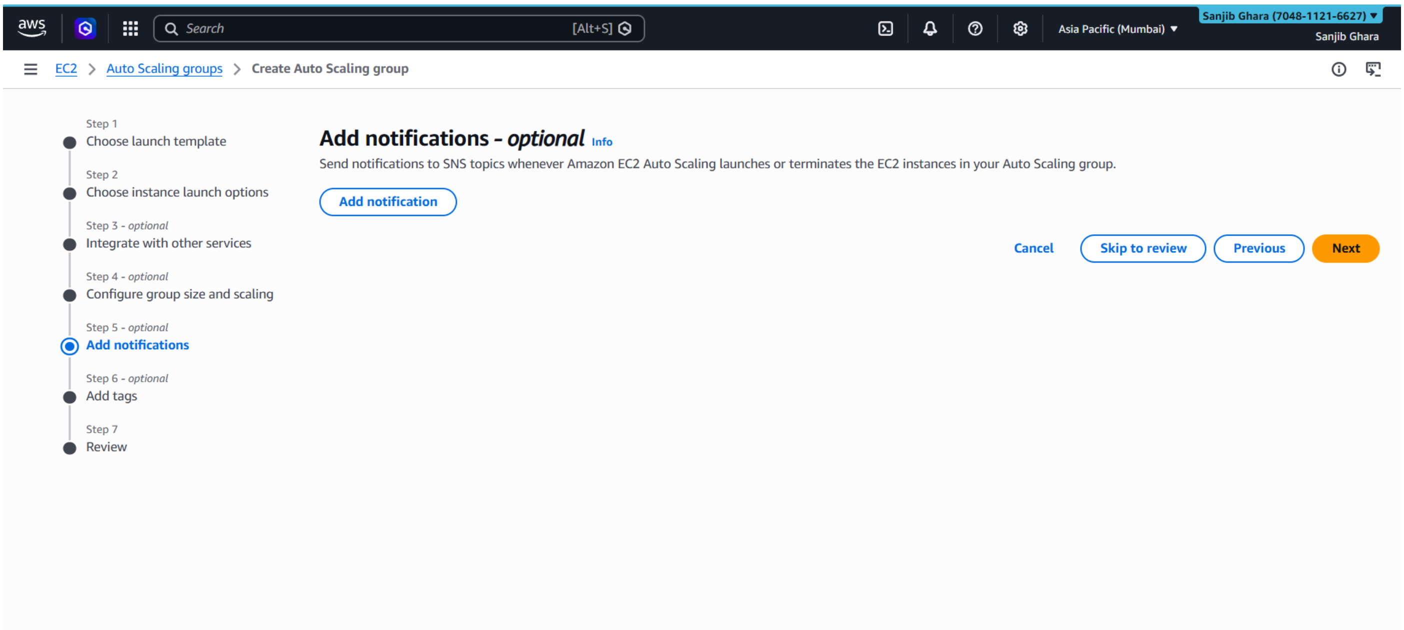
   - Instance warmup: **300** seconds. Click **Next.**

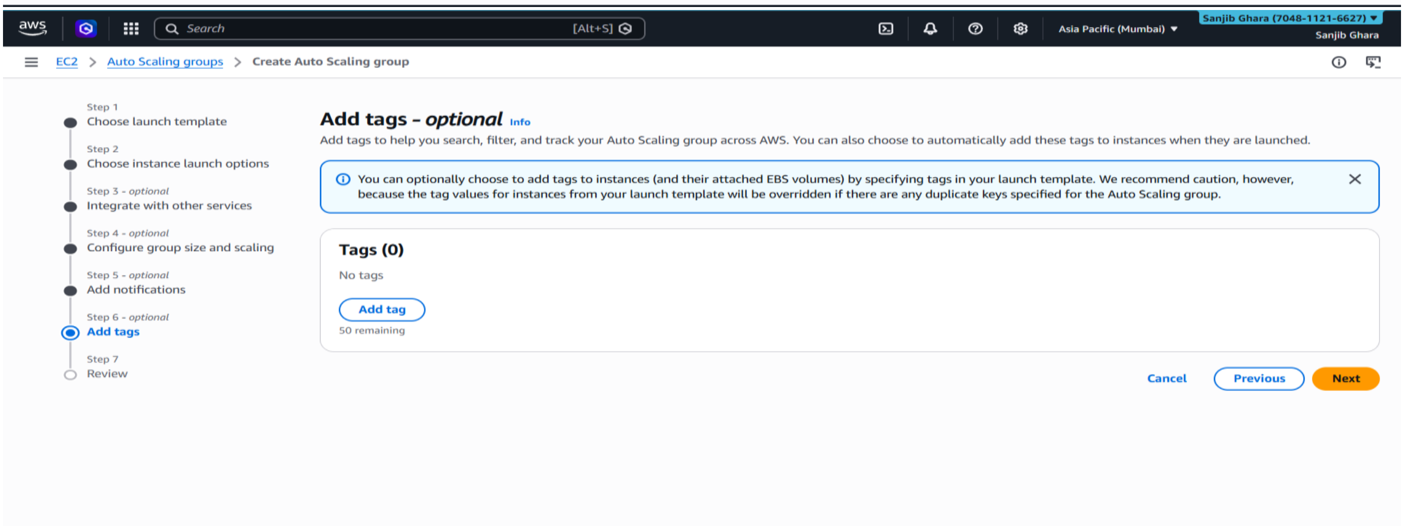


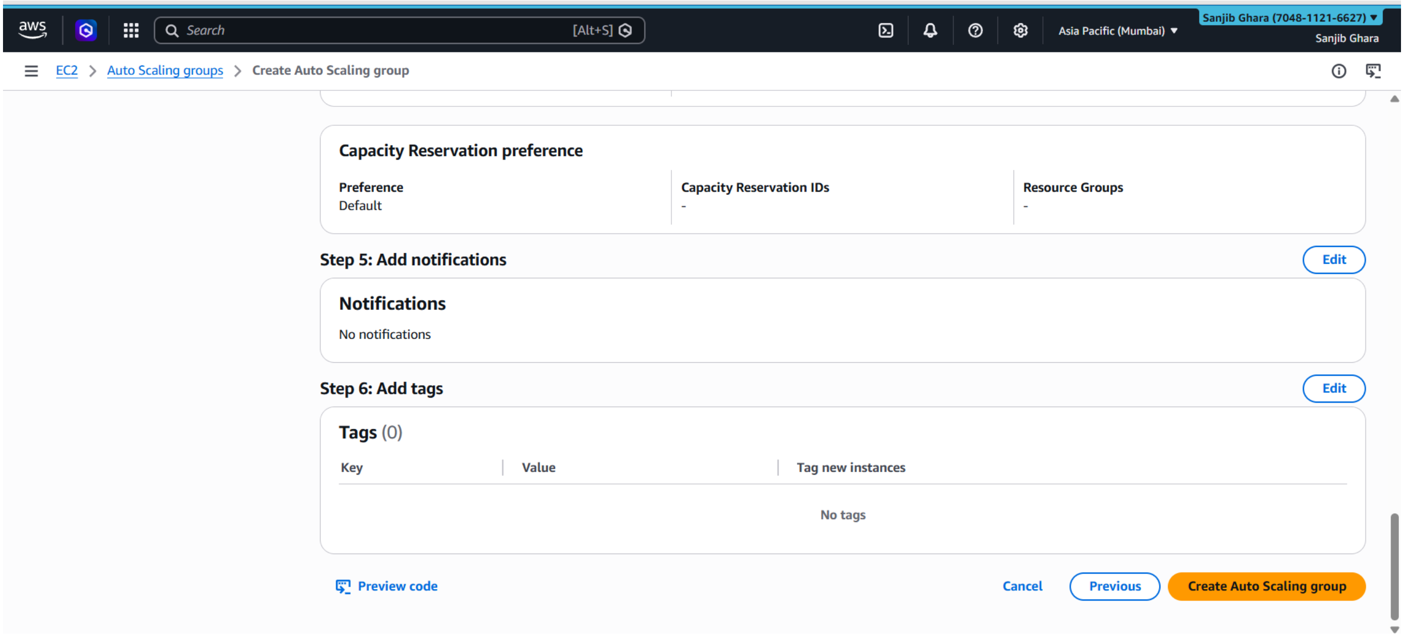




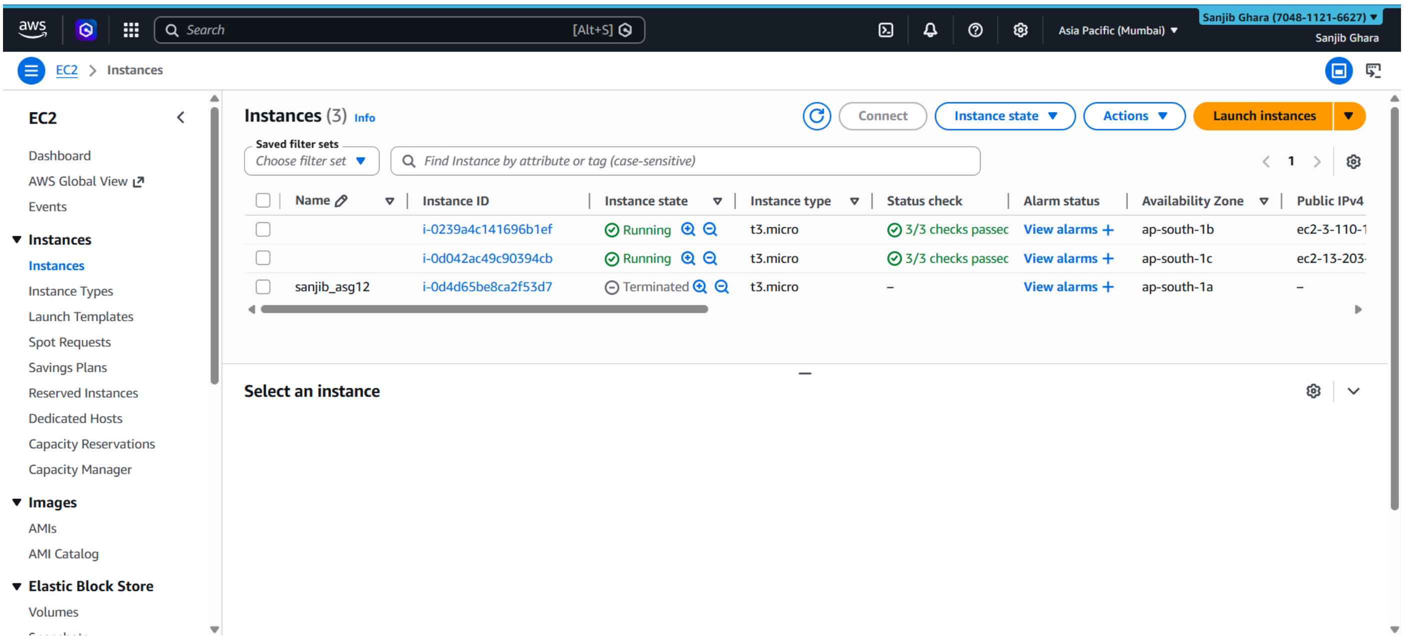
6. **Finalize:** Skip Step 5 (Notifications) and Step 6 (Tags) by clicking Next. Review your configurations and click **Create Auto Scaling group**.







After waiting some time….

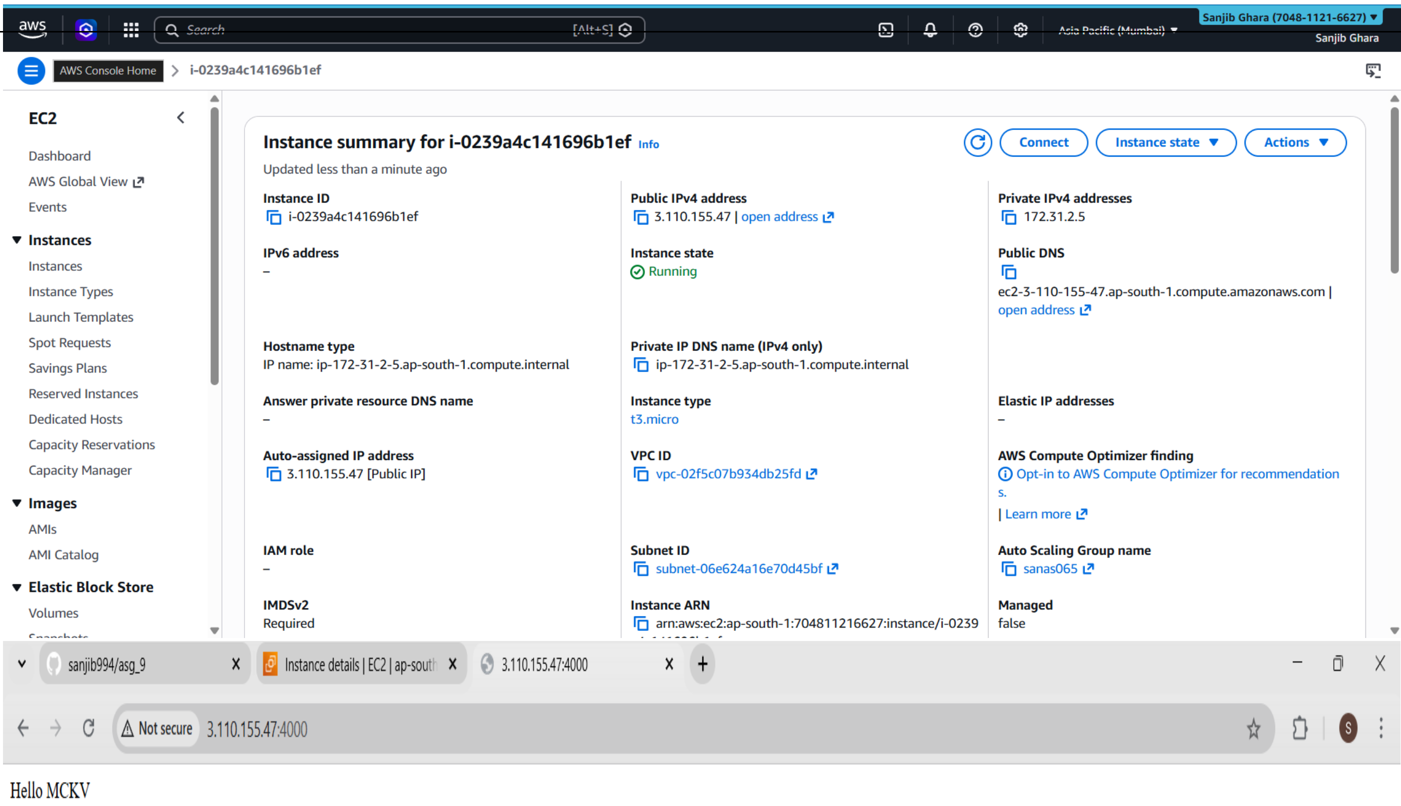


<u>Phase 3: Verify Application Deployment</u>

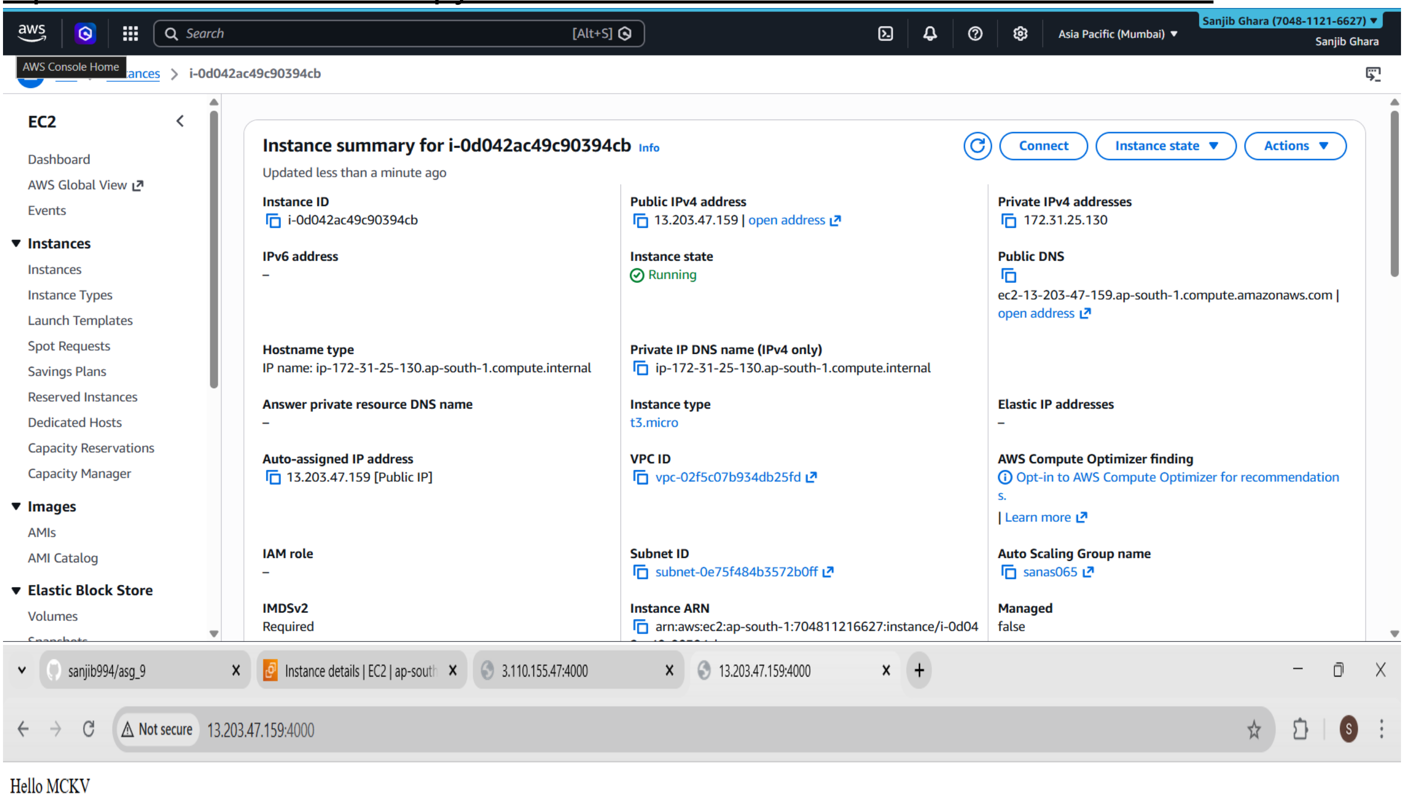
Ensure the ASG successfully launched instances and the Node.js app is running.

1. Go back to the **Instances** dashboard. I should see two instances in the "Initializing" or "Running" state.
2. Select the first instance, copy its **Public IPv4 address**, and paste it into a web browser followed by port 4000 (e.g., http://<Public-IP>:4000). It should display **"Hello MCKV"**.
3. Repeat the process for the second instance to verify it is also functioning correctly.

<u>Open 1st instance and Copy Public IPv4 address and Paste it in browser:</u>



Open 2nd instance and Copy Public IPv4 address and Paste it in browser:

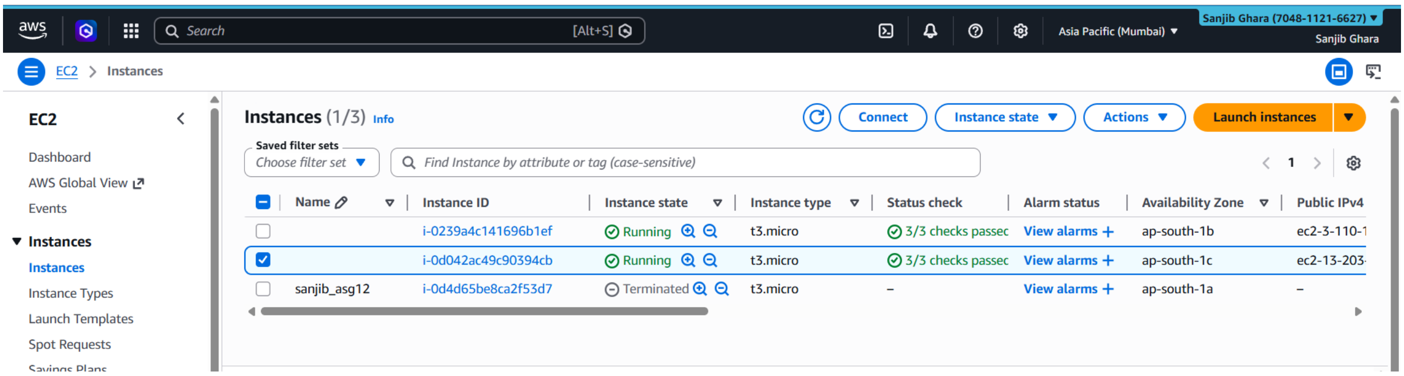


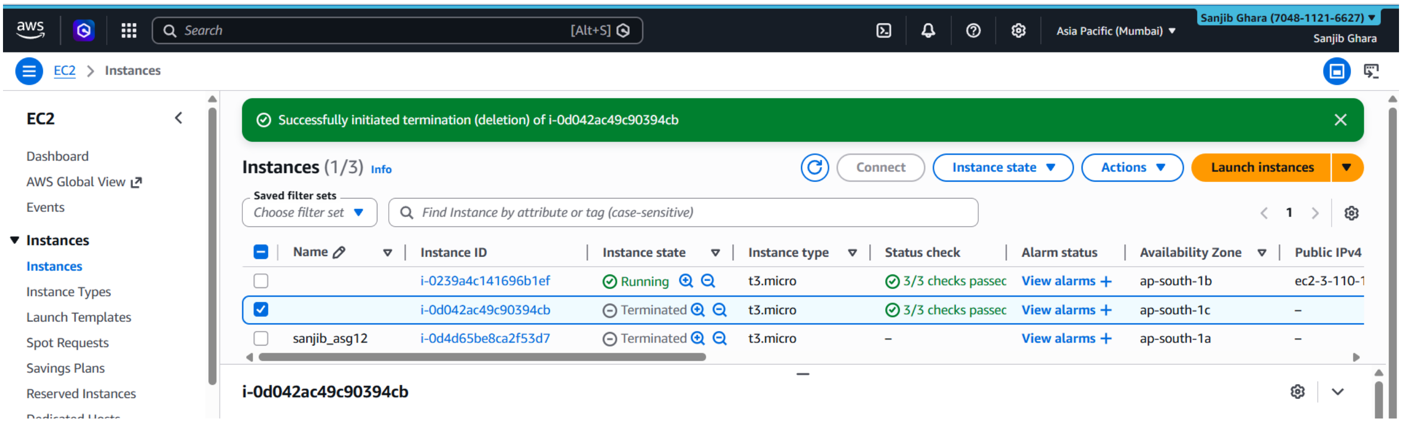
Phase 4: Test Auto-Recovery (Fault Tolerance)

Test if the ASG replaces crashed or terminated instances.

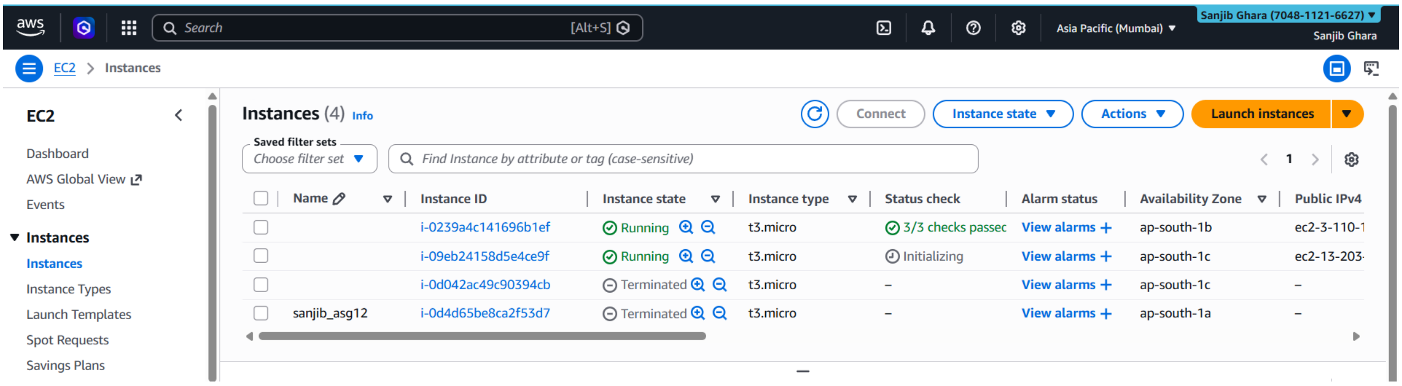
1. In the EC2 Instances dashboard, select one of the two running instances.
2. Go to **Instance state → Terminate (delete) instance** and confirm.
3. **Observation:** Wait a few moments. The Auto Scaling Group will detect that the instance count has fallen below the minimum desired capacity (2) and will

4. automatically initiate a new server to replace the terminated one.

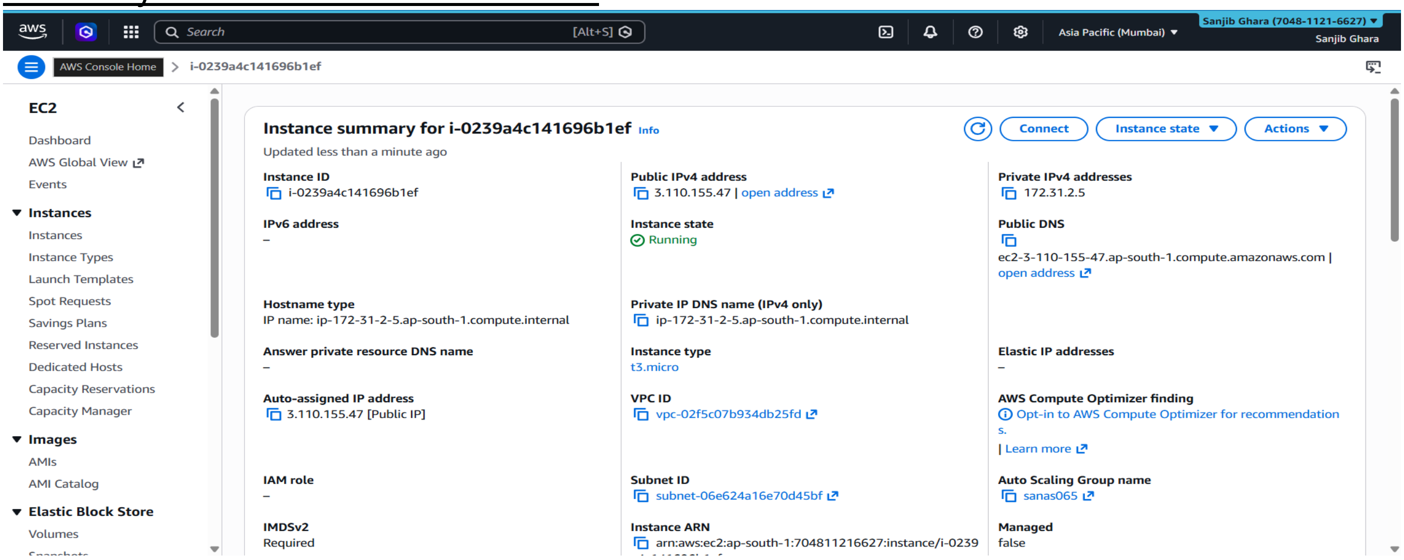




The Auto Scaling Group automatically provisions a replacement server after an instance fails or becomes overloaded.



Click any one of the two instances :



Phase 5: Test Scale-Out Policy (CPU Stress Test)

Artificially spike the CPU to trigger the target tracking scaling policy.

1. Select one of the running instances and connect to it using an SSH client (e.g., Bitvise SSH Client) using your .pem key.

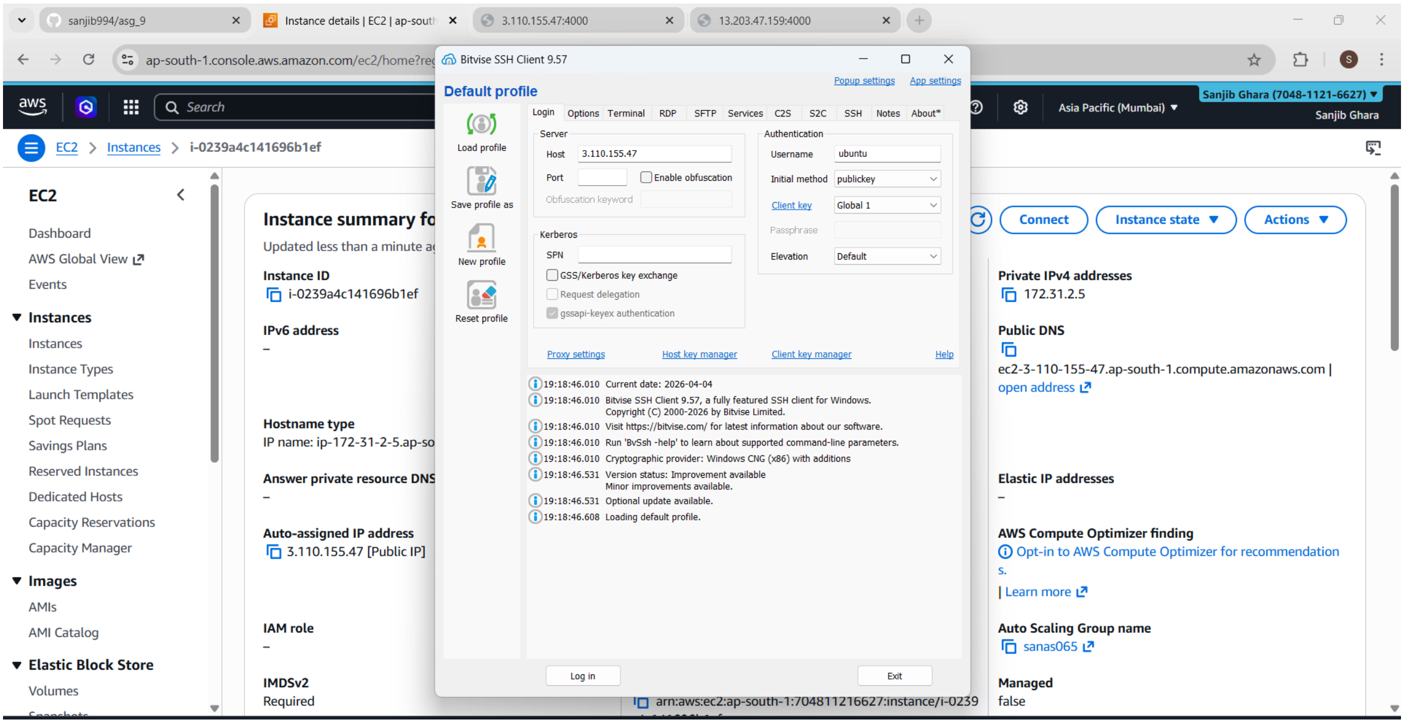
2. Accept the Host Key Verification prompt.
3. Open a new terminal console within Bitvise and create a script to stress the CPU:
   - Type nano file1.sh.
   - Write the following infinite loop script:

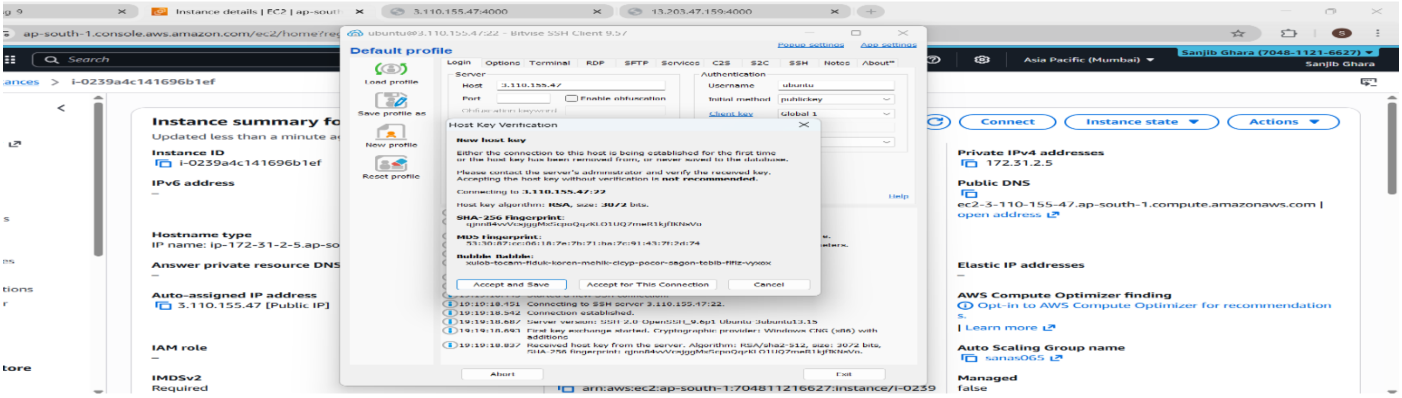
Bash

```
#!/bin/bash
while true
do
echo "Loop Running..."
done
```

   - Save and exit (Ctrl+O, Enter, Ctrl+X).

4. Make the script executable by running chmod +x file1.sh.
5. Execute the script by running ./file1.sh.
6. **Observation:** Go back to the AWS Console and check the EC2 metrics. You will notice the CPU utilization spiking significantly. Because it crosses your target value of 20%, the ASG will trigger an alarm and automatically launch a 3rd instance to handle the load.
7. Log out of the Bitvise SSH session to stop the script.





Open New Terminal Console:

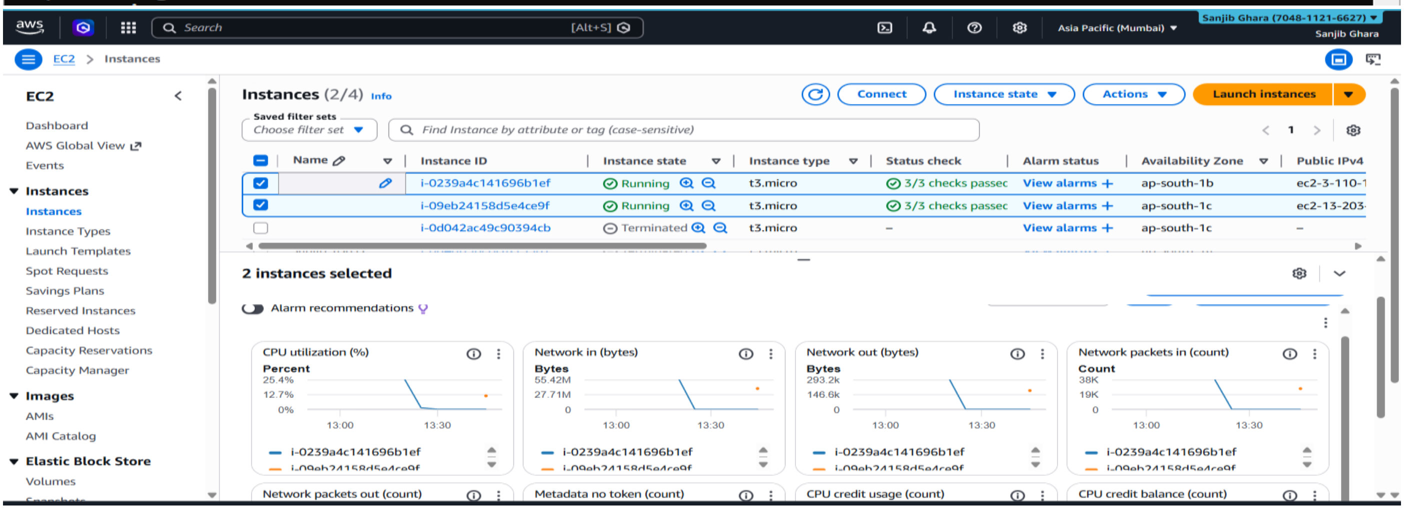
```
Last login: Sat Apr  4 13:49:43 2026 from 152.58.183.119
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-2-5:~$ nano file1.sh
```

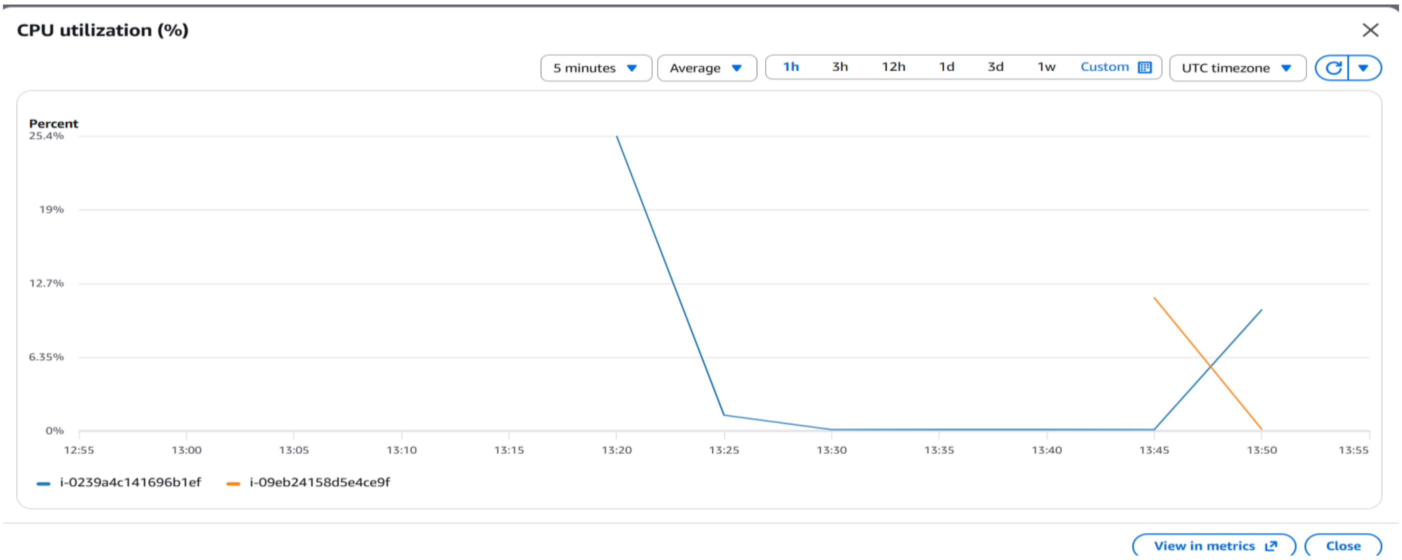
```
ubuntu@3.110.155.47:22 - Bitvise xterm - ubuntu@ip-172-31-2-5: ~
  GNU nano 7.2                         file1.sh *
#!/bin/bash
while true
do
echo "Loop Running..."
done
```

Press Ctrl +x to Exit…

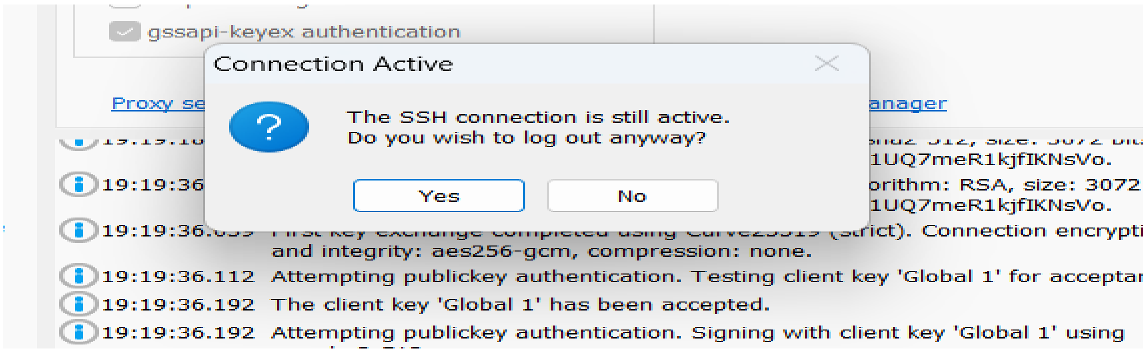
```
ubuntu@ip-172-31-2-5:~$ nano file1.sh
ubuntu@ip-172-31-2-5:~$ chmod +x file1.sh
ubuntu@ip-172-31-2-5:~$ ./file1.sh
```

```
ubuntu@3.110.155.47:22 - Bitvise xterm - ubuntu@ip-172-31-2-5: ~
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
Loop Running...
```





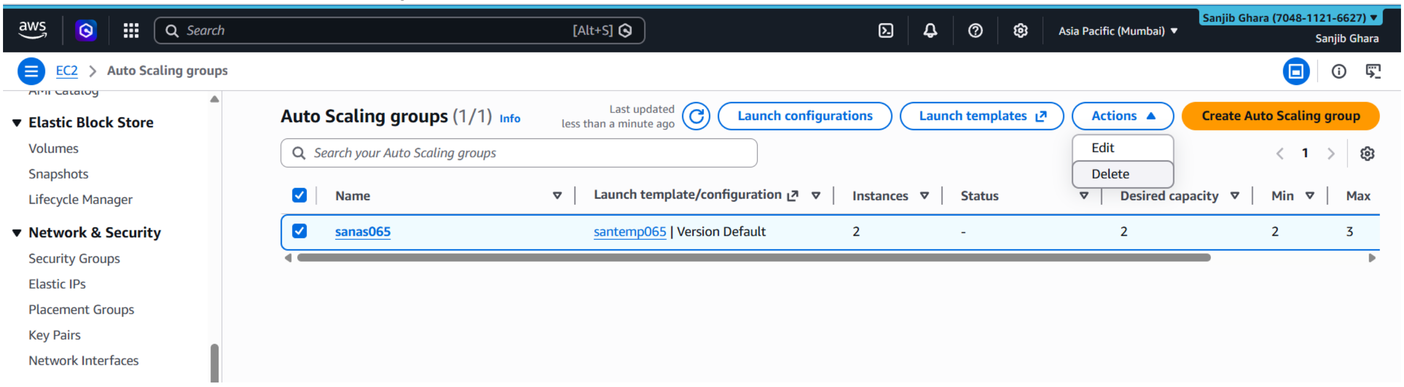
Now Log out from the bitvise SSH ..

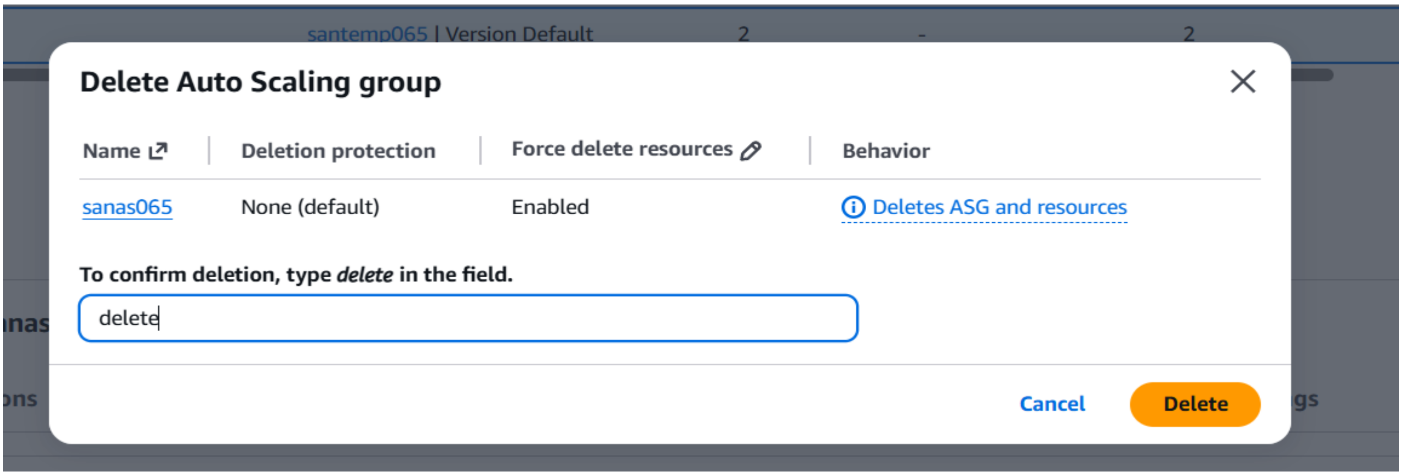


<u>Phase 6: Resource Cleanup</u>

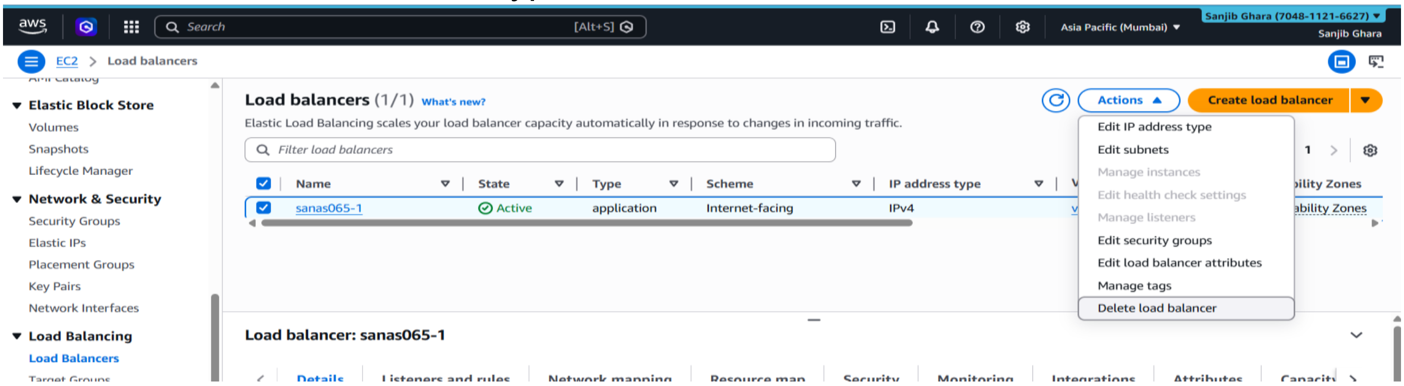
Crucial step to ensure you do not incur unwanted AWS charges.

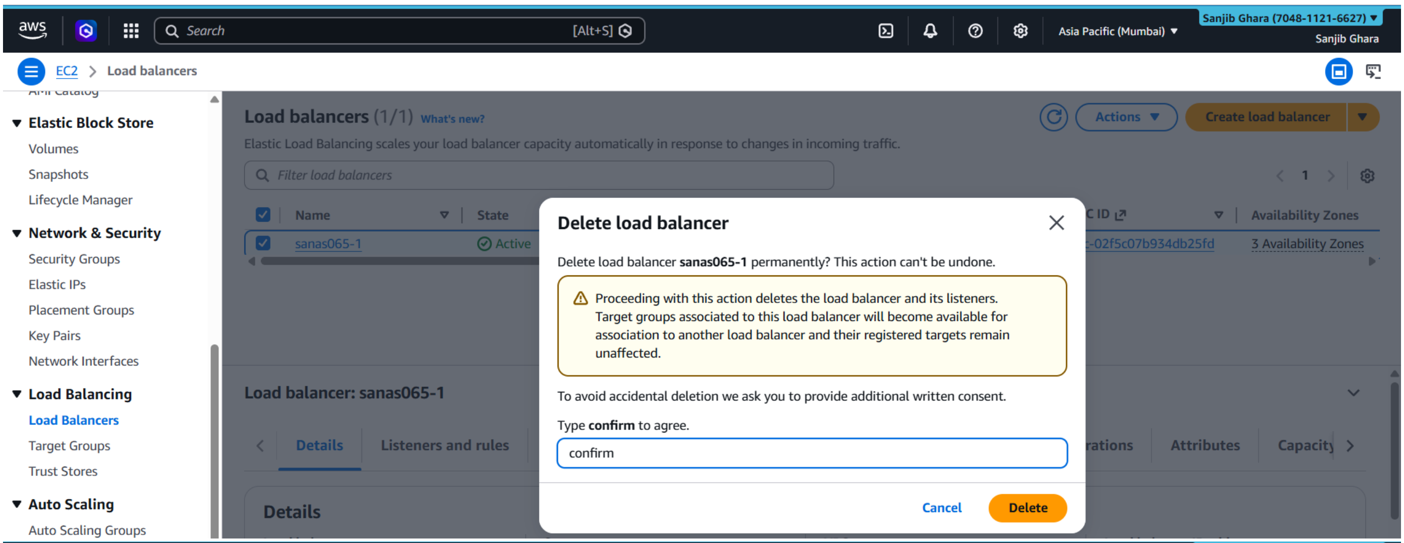
1. **Delete Auto Scaling Group:** Go to **Auto Scaling Groups**, select sanas065, click **Delete**, and type "delete" to confirm.

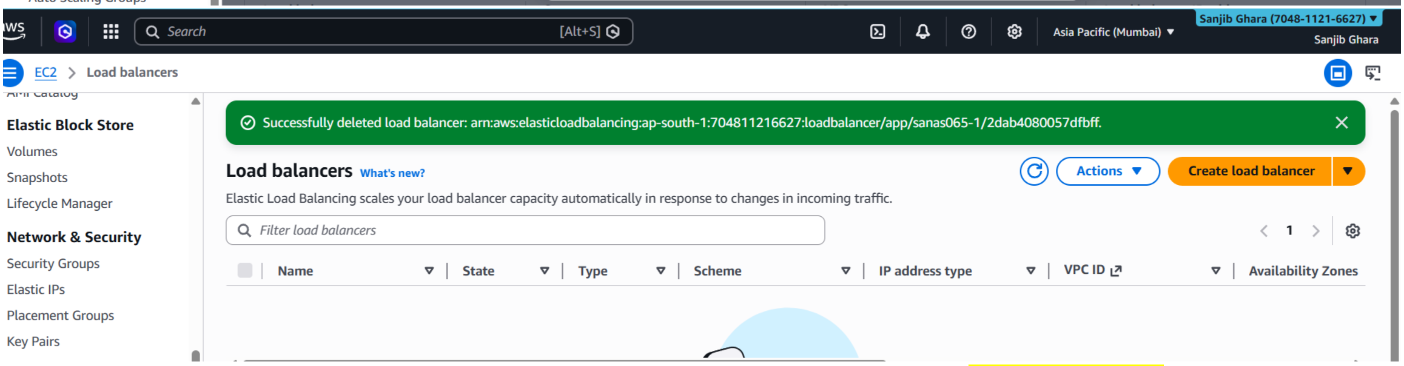




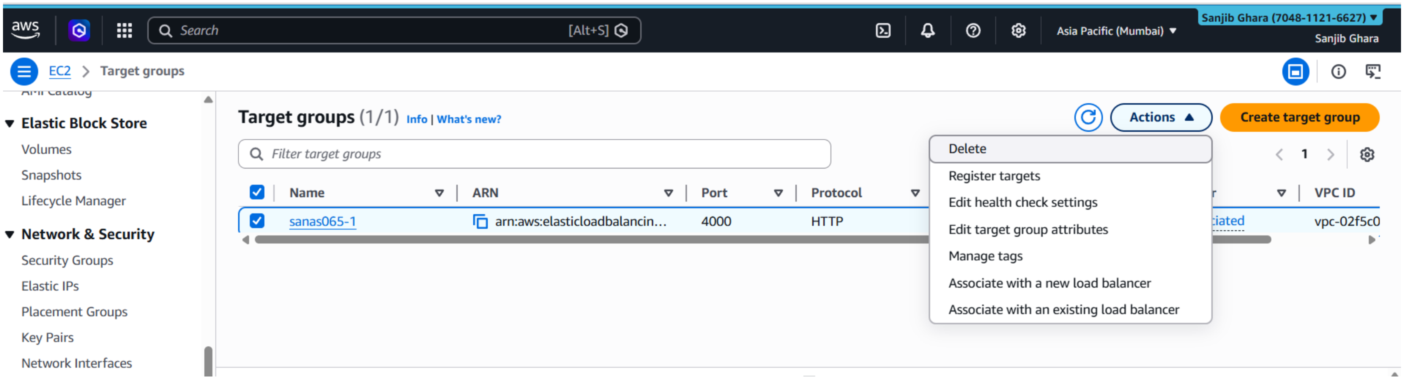
2. **Delete Load Balancer:** Go to **Load Balancers**, select sanas065-1, click **Actions → Delete**, and type "confirm".

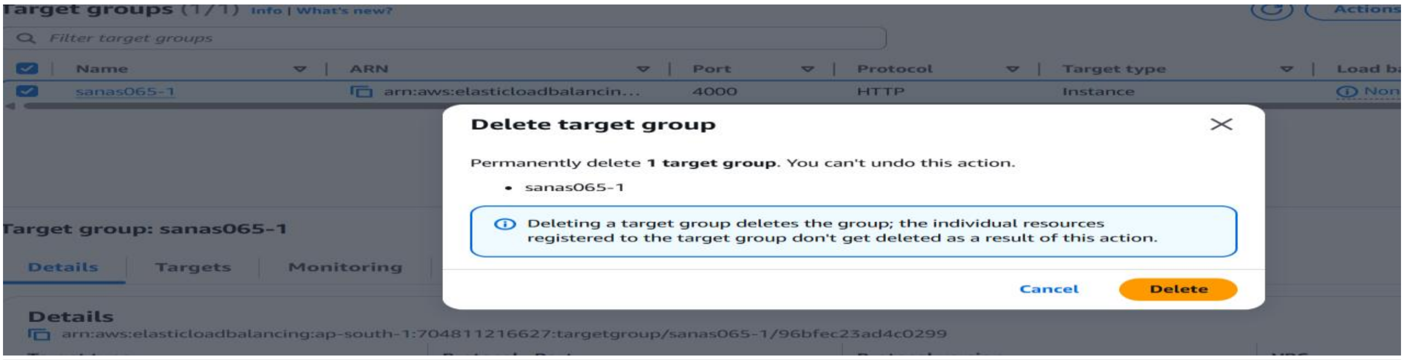


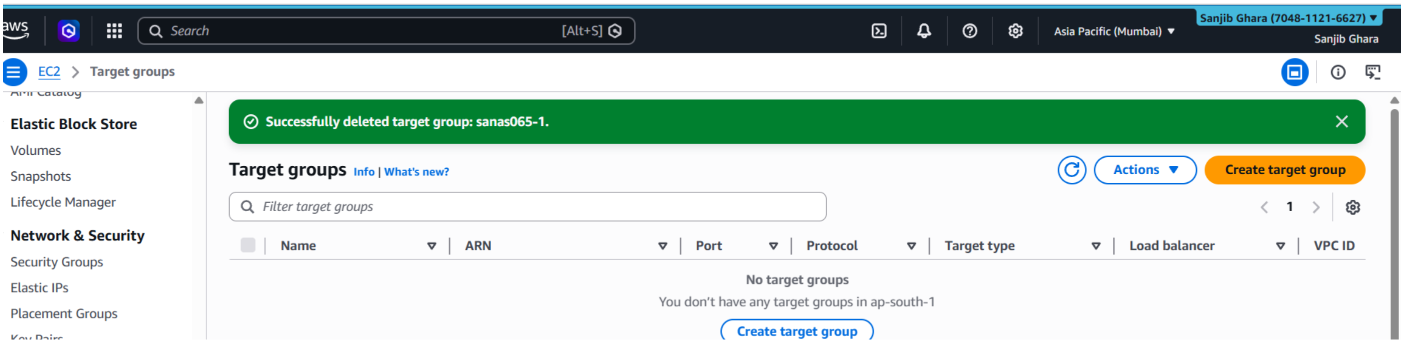




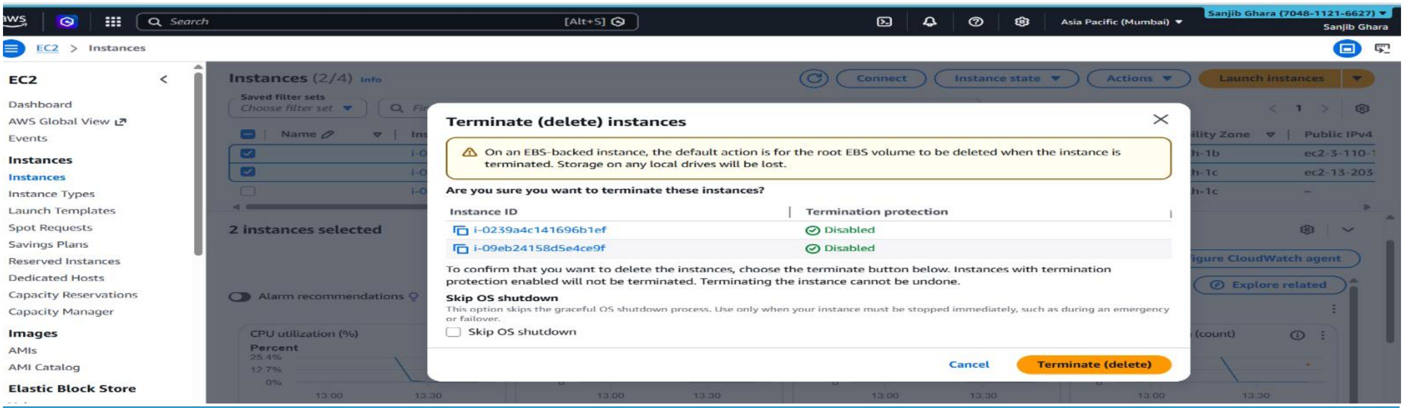
3. **Delete Target Group:** Go to **Target Groups**, select sanas065-1, click **Actions → Delete**, and confirm.

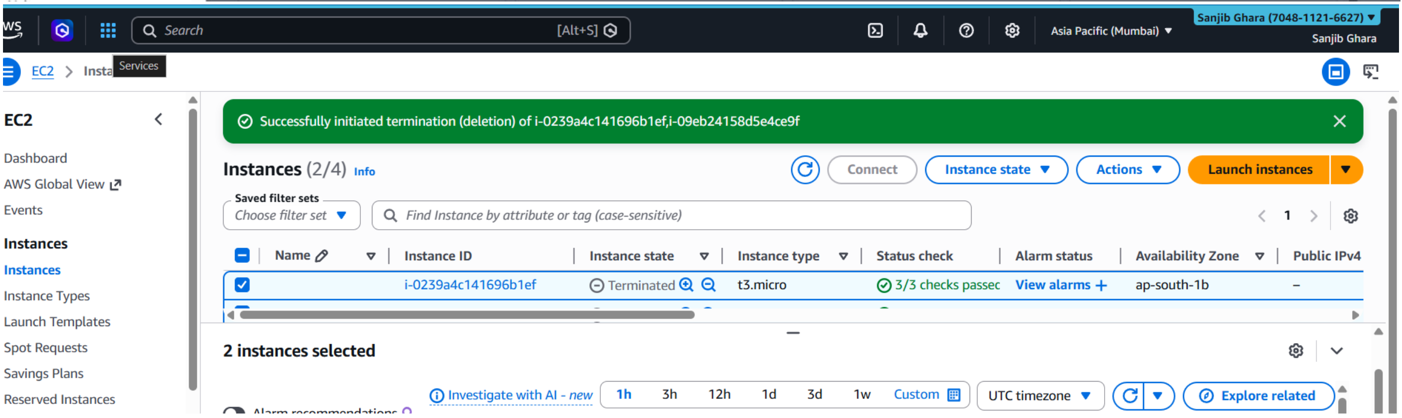






4.Terminate Instances: Go to the **Instances** dashboard, select any remaining running instances, click **Instance state → Terminate**, and confirm.





Conclusion

In this lab assignment, we successfully implemented and observed the behavior of AWS Auto Scaling and Application Load Balancing to balance the load across different EC2 instances. By configuring a Launch Template and an Auto Scaling Group, we achieved a highly available architecture capable of both self-healing and dynamic scaling.